

SUPSI

Rilevamento dati personali in documenti non strutturati aziendali

Studente/i

Algisi Gabriele

Relatore

Gay Mario

Correlatore

Consoli Angelo

Committente

Gay Mario

Corso di laurea

Ingegneria Informatica

Modulo

C11303

Anno

2025-2026

Data

13 agosto 2026

Indice

1	Introduzione	1
1.1	Riassunto / Abstract	1
1.2	Progetto assegnato	1
1.3	Contesto	2
1.4	Motivazione	3
1.5	Obiettivi del progetto	3
2	Analisi dei requisiti	5
2.1	Requisiti funzionali	5
2.1.1	Acquisizione dell'input	5
2.1.2	Estrazione del testo	5
2.1.3	Rilevamento delle PII	5
2.1.4	Aggregazione delle PII frammentate DA VALUTARE	6
2.1.5	Costruzione di una knowledge base delle entità DA VALUTARE	6
2.1.6	Ingestione del/i ROPA	6
2.1.7	Confronto dei risultati di analisi con il ROPA	7
2.1.8	Rilevamento delle anomalie di conformità	7
2.1.9	Generazione report per DPO	7
2.1.10	Persistenza e aggiornamento del registro delle PII rilevate	8
2.1.11	Feedback loop	8
2.1.12	Analisi contestuale della struttura del filesystem	8
2.1.13	Impieghi selettivi dell'AI generativa	8
2.2	Requisiti normativi	9
2.2.1	Conformità ai principi di limitazione delle finalità e della conservazione	9
2.2.2	Riferimento al registro delle attività di trattamento come fonte normativa	10
2.2.3	Copertura delle categorie di dati degni di particolare attenzione	10
2.2.4	Posizionamento rispetto all'art. 32(1)(d) GDPR	10
2.2.5	Esclusione delle basi giuridiche dal perimetro di analisi automatica	11
2.3	Requisiti operativi	12
2.3.1	Elaborazione locale dei dati	12

2.3.2	Vincolo sui modelli AI impiegabili	12
2.3.3	Facilità di installazione e aggiornamento	13
2.3.4	Estensibilità delle categorie e dei detector	13
2.3.5	Minimizzazione dei dati trattati dal sistema stesso	14
2.3.6	Operatività senza addestramento iniziale (zero-training)	14
2.4	Requisiti di qualità	15
2.4.1	Prestazioni su larga scala	15
2.4.2	Precisione del rilevamento	15
2.4.3	Tracciabilità delle anomalie	15
2.5	Requisiti di interfaccia	16
2.5.1	Interfaccia web come modalità primaria	16
2.5.2	Interfaccia a riga di comando (CLI)	17
2.5.3	Fruizione del report	17
3	Stato dell'arte	19
3.1	Background tecnologico	19
3.1.1	Spiegazione supervised vs unsupervised	19
3.1.2	Pre-tuning vs fine-tuning	20
3.1.3	Spiegazione zero-shot learning	20
3.2	Definizione e inquadramento del problema: Dalla Data Anonymization alla Data Governance	21
3.3	Compliance checking automatizzato	21
3.4	Approcci tecnici di PII detection	21
3.4.1	Recognition rule-based	21
3.4.2	Recognition con NLP - NER	22
3.4.3	Recognition con AI generativa	22
3.4.4	Limitazioni nella soluzione	22
3.4.5	Soluzioni ibride	23
3.4.6	Confronto empirico degli approcci	23
3.4.6.1	Prestazioni dei sistemi rule-based	23
3.4.6.2	Prestazioni dei modelli NER supervisionati	23
3.4.6.3	Prestazioni dei modelli NER zero-shot	24
3.4.6.4	Prestazioni dell'AI generativa	24
3.4.6.5	Convergenza verso la pipeline ibrida	24
3.4.6.6	Matrice di confronto sintetica	25
3.5	Analisi del training	25
3.5.1	Pre-training e fine-tuning (Paradigma classico)	25
3.5.2	Span-matching e Zero-shot (Il paradigma GLiNER)	26
3.6	Sfide nell'elaborazione documentale	26

3.6.1	Docling	26
3.6.2	Altro...	26
3.7	Panoramica dei sistemi esistenti	27
3.7.1	Sistemi open-source	27
3.7.2	Sistemi commerciali	27
3.8	Sintesi del gap e posizionamento del progetto	28
4	Analisi dei rischi	31
4.1	Rischi di progetto	32
4.1.1	Disponibilità e qualità dei dataset di valutazione	32
4.1.2	Complessità dell'integrazione multi-approccio	32
4.1.3	Costo computazionale e tempi di elaborazione	32
4.1.4	Dipendenza da modelli e librerie di terze parti	32
4.1.5	Carenza di competenze specialistiche	32
4.1.6	Scope creep e requisiti in evoluzione	32
4.2	Rischi di prodotto	32
4.2.1	Accuratezza insufficiente nella detection delle PII	32
4.2.2	Falsi positivi e impatto sull'usabilità	32
4.2.3	Falsi negativi e mancata rilevazione di PII critiche	32
4.2.4	Generalizzazione cross-domain (domain-silo problem)	32
4.2.5	Supporto multilingue e multilocalizzazione	32
4.2.6	Scalabilità su filesystem di grandi dimensioni	32
4.2.7	Qualità e completezza dei ROPA in input	32
4.2.8	Conformità normativa e aggiornamento dei framework legali	32
4.2.9	Sicurezza e protezione dei dati durante l'analisi	32
4.3	Valutazione dei rischi	32
4.3.1	Criteri di classificazione	32
4.3.2	Matrice probabilità–impatto	32
4.4	Strategie di mitigazione	32
5	Architettura del sistema	33
5.1	Ingestione e modello dati del ROPA	36
5.1.1	Strutturazione a monte o delega all'AI	36
5.1.2	Il registro-tipo CNIL come schema di riferimento	37
5.1.3	Normalizzazione in un modello relazionale	37
5.1.4	Ruolo circoscritto dell'AI: normalizzazione, non strutturazione	38
5.1.5	Punti aperti di progettazione	39
	5.1.5.0.1 Allineamento nLPD–GDPR.	39
	5.1.5.0.2 ROPA non strutturato.	39
5.2	Livello di estrazione del testo	39

5.2.0.0.1	Uno scope a stadi.	39
5.2.0.0.2	Estrazione e rilevamento come <i>concern</i> separati.	40
5.3	Livello di rilevamento ibrido delle PII	40
5.3.1	Due sorgenti complementari dietro un contratto unico	40
5.3.2	Il merge: unificazione e grado di certezza	40
5.3.2.0.1	Principio recall-first.	41
5.3.2.0.2	Un solo modello dato.	41
5.4	Persistenza del registro delle PII	41
5.4.1	Il requisito di tracciamento temporale	41
5.4.2	Approcci valutati	42
5.4.2.1	Opzione A — Snapshot completi	42
5.4.2.2	Opzione B — Event sourcing puro	42
5.4.2.3	Opzione C — Changelog ibrido delta-based	43
5.4.3	Motivazione della scelta	44
5.4.4	Modello dati concettuale	44
5.4.5	Risoluzione dei punti aperti di progettazione	45
5.4.5.0.1	Registrazione della «scomparsa» di una PII: esplicita.	45
5.4.5.0.2	Significato dell'evento «modifica»: scomposto, nessun MODIFIED.	46
5.4.5.0.3	Vocabolario chiuso dei tipi di variazione.	46
5.4.5.0.4	Ambito dello snapshot iniziale: prima scansione tutta NEW.	46
5.5	Associazione e verifica di conformità	47
5.5.1	Associazione documento–trattamento (B6)	47
5.5.2	Il confronto dichiarato–rilevato e il suo vocabolario (B7)	48
5.5.3	Verifica della conservazione	49
6	Implementazione	51
6.1	Ingestione del ROPA (blocco B1)	51
6.1.1	Il formato sorgente: il template CNIL reale	51
6.1.2	Lettura: un'unica forma per ODS ed Excel	51
6.1.3	Interpretazione: guidata dalle etichette, non dalle posizioni	52
6.1.4	Persistenza e mappatura assistita	52
6.2	Tecnologia di persistenza	53
6.3	Dati persistiti nei database	54
6.3.1	Registro ROPA (blocco B1)	54
6.3.1.0.1	processing_activity — l'attività di trattamento (livello 1).	54

6.3.1.0.2	macro_category — il blocco di categorie con la sua conservazione (livello 2).	54
6.3.1.0.3	declared_category — la categoria dichiarata, ponte verso il rilevamento (livello 3).	55
6.3.2	Registro delle PII rilevate (blocco B5)	55
6.3.2.0.1	registry_document — il documento analizzato.	55
6.3.2.0.2	registry_scan — una scansione, cioè un'osservazione nel tempo.	56
6.3.2.0.3	registry_pii_instance — la singola PII, stato corrente e mutabile.	56
6.3.2.0.4	registry_pii_change — il log append-only delle variazioni.	57
6.3.2.0.5	registry_folder_rule — la regola cartella→trattamento (blocco B6).	57
6.4	Riuso di un motore di detection esistente: valutazione di Microsoft Presidio	57
6.4.0.0.1	Profilo hardware.	58
6.4.1	Estensibilità config-driven: aggiungere categorie e pattern	59
6.5	Scelta dello Small Language Model per gli impieghi selettivi dell'AI	59
6.5.1	Criteri di selezione	60
6.5.2	Fattibilità dell'inferenza su CPU	60
6.5.3	Modelli candidati	61
6.5.3.0.1	Qwen2.5 (Alibaba).	61
6.5.3.0.2	Llama 3.2 1B/3B (Meta).	61
6.5.3.0.3	Phi-3-mini / Phi-4-mini (Microsoft).	61
6.5.3.0.4	Gemma 3 (Google).	61
6.5.3.0.5	SmolLM2 (Hugging Face).	61
6.5.3.0.6	IBM Granite 3.x.	62
6.5.4	Analisi per task	62
6.5.4.0.1	Task 1 — ROPA verso categorie.	62
6.5.4.0.2	Task 2 — Analisi di documenti campione.	62
6.5.4.0.3	Task 3 — Struttura del file system.	62
6.5.5	Confronto e conclusione	62
6.5.6	Energia, efficienza e limiti	63
6.6	Deployment containerizzato e portabilità	64
6.6.1	Portabilità e isolamento delle dipendenze	64
6.6.2	Architettura di deployment	64
6.6.3	Contratti e direzione	65

STUDENT SUPSI

Capitolo 1

Introduzione

1.1 Riassunto / Abstract

Qui ci sarà l'abstract...

1.2 Progetto assegnato

Le aziende e le organizzazioni soggette alla nLPD (nuova Legge federale sulla protezione dei dati) sono tenute a sapere dove si trovano i dati personali che trattano, a quale trattamento riportato del registro dei trattamenti sono riconducibili e se sono ancora entro i termini di conservazione dichiarati nel registro. Nella pratica però, gran parte di questi dati è dispersa in documenti non strutturati — ad esempio PDF, Word, Excel, scansioni — senza che l'organizzazione ne abbia visibilità sistematica.

Gli strumenti disponibili sul mercato, sia quelli più abordabili (es. PII Crawler, PII Tools) sia quelli enterprise (es. BigID, Varonis), sono progettati per garantire la conformità con normative europee (GDPR) o statunitensi (es.: CCPA) e non necessariamente gestiscono le categorie di dati specifiche della nLPD svizzera senza configurazioni e validazioni specifiche. Oltre a ciò, solo alcuni prodotti enterprise più avanzati integrano il registro dei trattamenti come input normativo configurabile. Di conseguenza la verifica del superamento del periodo di retention è spesso possibile solo tramite una categorizzazione molto precisa all'origine.

Il progetto propone lo sviluppo di un sistema in grado di evidenziare il divario tra le attività di trattamento dichiarate nel registro e l'effettiva gestione dei documenti aziendali, anche utilizzando tecniche NLP e AI. Per ciascun documento analizzato, il sistema deve identificare eventuali dati personali presenti (Personally Identifiable Information, PII), associare il documento alla corrispondente attività di trattamento combinando diversi indicatori (es. posizione nel file system, contenuto, PII identificati) e verificare la conformità al periodo di conservazione dichiarato nel registro. I documenti che hanno superato il loro periodo

di conservazione e le PII [1] che non possono essere collegate ad alcuna operazione di trattamento registrata devono essere segnalati come anomalie.

1.3 Contesto

La protezione dei dati personali costituisce oggi uno dei nodi centrali della governance organizzativa, tanto sul piano giuridico quanto su quello operativo. In Europa, il regolamento (UE) 2016/679 (GDPR) [2] ha introdotto un corpus di obblighi sostanziali per i titolari del trattamento, tra cui il principio di limitazione della conservazione sancito dall'art. 5(1)(e), secondo cui i dati personali devono essere conservati "in una forma che consenta l'identificazione degli interessati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati". In Svizzera, la nuova Legge federale sulla protezione dei dati (nLPD, RS 235.1, in vigore dal 1° settembre 2023) ha recepito principi analoghi: l'art 6(4) dispone che "i dati personali sono distrutti o resi anonimi appena non sono più necessari per lo scopo del trattamento", stabilendo un legame diretto tra la finalità dichiarata e il ciclo di vita del dato. Entrambe le normative impongono inoltre la tenuta di un registro delle attività di trattamento (ROPA, Record of Processing Activities). Il GDPR, all'art. 30(1)(f), richiede che il registro contenga, "ove possibile, i termini ultimi previsti per la cancellazione delle diverse categorie di dati"; la nLPD [3], all'art. 12(2)(e), prevede analogamente l'indicazione della durata di conservazione dei dati personali o dei criteri per determinarne la durata. Il ROPA costituisce dunque il documento normativo di riferimento per qualsiasi verifica di conformità: è in esso che l'organizzazione dichiara cosa tratta, perché, con quale base giuridica e per quanto tempo.

I due ordinamenti divergono tuttavia su un punto strutturalmente rilevante per qualsiasi sistema di verifica automatica. Il GDPR enumera positivamente le basi giuridiche lecite all'art. 6(1), subordinando la liceità del trattamento alla sussistenza di almeno una di esse. La nLPD adotta invece un approccio invertito: il trattamento è lecito per default in quanto esercizio della libertà d'azione dei privati, ma diventa illecito se lede ingiustificatamente la personalità dell'interessato ai sensi dell'art. 30. La giustificazione può provenire dal consenso, da un interesse preponderante o dalla legge, secondo il catalogo dell'art. 31. Questa differenza strutturale impone che un sistema di analisi della conformità tenga conto del framework giuridico applicabile, poiché il criterio con cui si valuta l'esistenza di una base legittima non è identico nei due contesti.

Nella pratica organizzativa, la conformità dichiarata nel registro tende a divergere dalla situazione reale dei dati. Gran parte dell'informazione trattata da un'organizzazione è incorporata in documenti non strutturati (file PDF, Word, Excel, messaggi di posta elettronica...) distribuiti su file system locali, server condivisi e servizi cloud. Questi documenti non recano metadati normalizzati che consentono di associarli automaticamente a un'attività di trattamento specifica, né di verificarne la conformità al periodo di retention dichiarato. Il risultato è

un divario strutturale tra ciò che il registro descrive e ciò che esiste effettivamente nei sistemi informativi dell'organizzazione.

Questo divario assume rilevanza giuridica diretta. Il GDPR all'art. 32(1)(d) qualifica come misura di sicurezza obbligatoria una procedura per testare, verificare e valutare regolarmente l'efficacia delle misure tecniche e organizzative al fine di garantire la sicurezza del trattamento. La nLPD, pur con formulazione più sintetica, impone all'art. 8(1) che il titolare e il responsabile del trattamento garantiscano, mediante appropriati provvedimenti tecnici e organizzativi, che la sicurezza dei dati personali sia adeguata al rischio. Un sistema in grado di rilevare automaticamente i dati personali presenti nei documenti aziendali e di verificarne la coerenza con il registro può essere inteso come implementazione tecnica di quest'obbligo.

1.4 Motivazione

Il divario descritto nella sezione precedente tra le attività di trattamento dichiarate nel registro e la gestione effettiva dei documenti non è soltanto un problema teorico: ha conseguenze dirette sulla capacità di un'organizzazione di dimostrare conformità. Un Data Protection Officer che debba rispondere a una richiesta dell'autorità di controllo, o semplicemente verificare se un'attività di trattamento è ancora coperta da una fase giuridica valida, si trova nella pratica privo di uno strumento che colleghi sistematicamente il contenuto dei documenti aziendali al registro. La verifica, quando viene fatta è perlopiù manuale, parziale e non ripetibile nel tempo.

Come verrà approfondito nell'analisi degli strumenti esistenti ([LINK A SEZIONE]), il panorama commerciale attuale non risolve in modo sufficiente questo problema per il contesto svizzero, lasciando aperto uno spazio applicativo concreto. Allo stesso tempo, come discusso nella motivazione architettuale ([LINK A SEZIONE]), la maturità raggiunta dalle tecniche di elaborazione del linguaggio naturale rende oggi percorribile, a un costo computazionale sostenibile, un approccio che pochi anni fa avrebbe richiesto un investimento sproporzionato.

Il progetto nasce dunque dall'intersezione di tre fattori: un obbligo normativo concreto, non pienamente automatizzabile con gli strumenti esistenti; uno spazio applicativo non coperto in modo adeguato dal mercato; e una maturità tecnologica che rende oggi realistico costruire un sistema di verifica della conformità basato sul confronto tra registro dichiarato e contenuto documentale effettivo.

1.5 Obiettivi del progetto

Per ora solamente obiettivi del progetto, valutare se riformulare

Il progetto intende produrre un sistema che renda visibile lo scarto tra le attività di trattamento dichiarate nel registro e la gestione effettiva dei dati nei documenti aziendali. In pratica, il prototipo dovrà:

- rilevare le categorie di dati personali presenti in documenti non strutturati, sulla base delle categorie canoniche della nLPD e di quelle dichiarate nel registro dei trattamenti;
- associare i documenti al trattamento corrispondente combinando più segnali;
- verificare la conformità ai periodi di retention dichiarati nel registro;
- segnalare categorie di dati personali non dichiarate nel registro e trattamenti non censiti;
- produrre un report leggibile dal Data Protection Officer dell'organizzazione.

Capitolo 2

Analisi dei requisiti

La struttura di questo capitolo si ispira allo standard ISO/IEC/IEEE 29148:2018 - Systems and Software Engineering - Life cycle processes - Requirements engineering [4].

2.1 Requisiti funzionali

2.1.1 Acquisizione dell'input

Il sistema deve poter analizzare come input un insieme di file selezionati, una cartella, oppure un intero filesystem. L'acquisizione deve avvenire senza presupporre una struttura predefinita dell'albero di directory, supportando l'elaborazione ricorsiva di sottocartelle.

L'utente deve poter specificare percorsi multipli e/o pattern di esclusione (es. cartelle di sistema, file temporanei) per limitare la scansione ai soli percorsi rilevanti.

2.1.2 Estrazione del testo

Il sistema deve estrarre il contenuto testuale dai documenti non strutturati (PDF, Word, Excel, immagini scansionate) [5] indipendentemente dal formato di partenza.

L'estrazione deve preservare, dove possibile, i metadati strutturali del documento (es. tabelle, intestazioni, numeri di pagina) necessari a contestualizzare le PII rilevate e a tracciarne la posizione nel report finale (cfr. 2.4.3).

2.1.3 Rilevamento delle PII

Il sistema deve identificare le occorrenze di dati personali nel testo estratto, tramite una combinazione di meccanismi (regex, ML, NLP) [6, 7, 8] la cui composizione esatta è oggetto di definizione architetturale.

Il rilevamento deve coprire, come minimo, le categorie di dati personali rilevanti ai sensi della nLPD e del GDPR (es. dati identificativi, dati di contatto, identificatori numerici nazionali, dati

relativi alla salute, dati degni di particolare protezione secondo l'art. 5 lett. c nLPD [3] e l'art. 9 GDPR [2]).

2.1.4 Aggregazione delle PII frammentate | DA VALUTARE

Il sistema deve essere in grado di ricondurre frammenti di informazione dispersi all'interno di uno o più documenti a un'unica entità coerente (es. nome, identificativo, indirizzo di una stessa persona riportati in punti diversi del testo o in documenti diversi).

L'aggregazione deve basarsi su criteri verificabili (es. co-occorrenza, similarità semantica, identificatori univoci condivisi) e deve essere documentata nel report ai fini di tracciabilità.

2.1.5 Costruzione di una knowledge base delle entità | DA VALUTARE

A partire dalle PII estratte, il sistema deve costruire una rappresentazione strutturata delle entità rilevate nel sistema documentale (tramite NER [9]), che costituisce la base per il confronto con il ROPA.

La rappresentazione deve associare a ciascuna entità: le categorie di PII rilevate, i documenti di provenienza, e i riferimenti di posizione necessari per l'auditing.

2.1.6 Ingestione del/i ROPA

Il sistema deve accettare in input uno o più registri delle attività di trattamento (ROPA) in formato strutturato, da utilizzare come riferimento normativo autoritativo per il confronto con le PII rilevate nei documenti.

Razionale. Il ROPA costituisce, ai sensi dell'art. 30 GDPR e dell'art. 12 nLPD, il documento in cui l'organizzazione dichiara le proprie attività di trattamento. Senza la sua ingestione, il sistema non potrebbe effettuare alcuna verifica di conformità.

Il sistema deve supportare almeno i formati tabulari di uso comune (CSV, XLSX, JSON). Per ogni attività di trattamento, il formato di ingestione deve includere almeno i seguenti campi:

- categorie di dati personali previste;
- finalità del trattamento;
- periodo di conservazione dichiarato (o criteri per la sua determinazione);
- categorie di interessati;
- eventuali categorie di destinatari.

Il sistema deve segnalare all'utente, in fase di ingestione, eventuali campi obbligatori mancanti o valori non interpretabili, senza interrompere l'elaborazione per i record validi.

Criterio di verifica. Il requisito è soddisfatto se il sistema è in grado di importare correttamente un ROPA di riferimento in ciascuno dei formati supportati, estraendone tutti i campi previsti e segnalando i record incompleti.

2.1.7 Confronto dei risultati di analisi con il ROPA

Il sistema deve confrontare le entità e le categorie di dati personali effettivamente rilevate nei documenti con quanto dichiarato nel/i ROPA, verificando l'accuratezza e la coerenza tra dati trovati e dati dichiarati.

Il confronto deve produrre, per ciascuna entità/documento analizzato, un'associazione (anche parziale o nulla) con una o più attività di trattamento del ROPA, secondo criteri ed euristiche definiti in fase di progettazione architettuale.

2.1.8 Rilevamento delle anomalie di conformità

Il sistema deve identificare e classificare, sulla base del confronto di cui alla sezione 2.1.7, le seguenti tipologie di anomalie:

- categorie di PII rilevate ma non dichiarate nel ROPA per l'attività di trattamento associata;
- dati personali privi di una base giuridica/attività di trattamento associabile ("PII orfane");
- violazioni dei periodi di conservazione dichiarati nel ROPA (documenti contenenti PII oltre il termine di retention dichiarato).

Ogni anomalia rilevata deve essere classificata secondo una tipologia predefinita e associata a un livello di gravità/priorità.

2.1.9 Generazione report per DPO

Il sistema deve produrre, come esito dell'analisi, un report di conformità destinato al Data Protection Officer (DPO), che raccolga le anomalie rilevate e sia consultabile senza competenze tecniche.

Il report deve includere, per ciascuna anomalia: tipologia, documento/i coinvolti, attività di trattamento di riferimento (se presente) e la motivazione del rilevamento. Deve inoltre essere organizzato **per documento**, in modo che il DPO possa passare dal quadro d'insieme del corpus documentale al dettaglio del singolo file, delle sue PII e del suo verdetto.

Vincolo di minimizzazione. Il report deve descrivere le PII rilevate come *riferimenti* — categoria, posizione, provenienza del rilevamento — e non deve in nessun caso riportare il valore del dato personale rilevato (cfr. 2.3.5).

2.1.10 Persistenza e aggiornamento del registro delle PII rilevate

Il sistema deve mantenere un registro interno persistente delle PII rilevate, che associ a ciascuna occorrenza: il file sorgente, la posizione all'interno del documento, la categoria di PII e un identificatore temporale dell'analisi.

Il registro deve consentire il confronto tra esecuzioni successive dello stesso corpus documentale, al fine di rilevare:

- PII precedentemente segnalate che risultano rimosse o modificate nel documento sorgente (es. a seguito di un intervento correttivo da parte dell'organizzazione);
- nuove PII introdotte da documenti aggiunti o modificati dopo l'ultima analisi;
- documenti eliminati dal filesystem che contenevano PII precedentemente censite.

Il formato del registro deve essere strutturato e leggibile meccanicamente (es. database locale, JSON), per supportare sia la consultazione diretta sia l'integrazione con il report di conformità.

2.1.11 Feedback loop

Richiesto inserire un processo di iterazione tra i dipendenti e il DPO (gestito dal sistema), per segnalare all'interno dei documenti aziendali delle PII sfuggite al rilevamento automatico.

2.1.12 Analisi contestuale della struttura del filesystem

Il sistema deve considerare la posizione di ciascun documento all'interno della gerarchia del filesystem come segnale contestuale per l'associazione all'attività di trattamento. La struttura delle cartelle (nomi, profondità, organizzazione logica) può fornire indicazioni sulla finalità del trattamento e sulla legittimità della presenza di determinati dati personali in una posizione specifica.

A titolo esemplificativo, un documento contenente dati personali di un candidato (es. curriculum) collocato in una cartella destinata alla gestione dei turni deve essere segnalato come anomalia posizionale nel report, in quanto indicativo di una possibile violazione del principio di limitazione delle finalità.

2.1.13 Impieghi selettivi dell'AI generativa

Il sistema può impiegare modelli di AI generativa per attività mirate che beneficiano della comprensione semantica profonda, senza richiedere l'applicazione sistematica a ogni documento del corpus. In particolare:

- **Arricchimento dei ROPA:** qualora il registro delle attività di trattamento fornito in input contenga descrizioni sintetiche o generiche delle categorie di dati (es. “dati anagrafici” anziché “nome, cognome, data di nascita, indirizzo”), il sistema può utilizzare un LLM per espandere e normalizzare tali descrizioni [10], migliorando la granularità del confronto con le PII rilevate;
- **Analisi campionata:** il sistema può sottoporre a un LLM un sottoinsieme rappresentativo di documenti (es. uno ogni cento) per ottenere un’analisi contestuale approfondita, identificando PII deducibili dal contesto che sfuggirebbero ai metodi rule-based e NER;
- **Analisi della struttura del filesystem:** il sistema può utilizzare un LLM per interpretare la semantica dei percorsi e dei nomi di cartella, al fine di rilevare incoerenze tra la posizione del documento e il suo contenuto (cfr. 2.1.12).

Questi impieghi devono essere documentati e giustificati da studi comparativi (cfr.10) che ne attestino il valore aggiunto rispetto al costo computazionale sostenuto.

2.2 Requisiti normativi

2.2.1 Conformità ai principi di limitazione delle finalità e della conservazione

Il sistema deve supportare la verifica di conformità ai seguenti principi normativi:

- **Limitazione della finalità** (art. 5(1)(b) GDPR [2]; art. 6 cpv. 3 nLPD [3]): i dati personali devono essere raccolti per finalità determinate, esplicite e legittime, e non ulteriormente trattati in modo incompatibile con tali finalità;
- **Limitazione della conservazione** (art. 5(1)(e) GDPR [2]; art. 6 cpv. 4 nLPD [3]): i dati personali devono essere conservati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati.

Razionale. Questi due principi costituiscono il fondamento giuridico dell’intera logica di verifica del sistema: il confronto tra le PII rilevate nei documenti e quanto dichiarato nel ROPA è, nella sua essenza, una verifica che i dati trattati siano coerenti con le finalità dichiarate e che non siano conservati oltre i termini previsti.

La verifica deve operare su due assi:

1. **Asse delle finalità:** per ciascun documento analizzato, il sistema deve verificare che le categorie di PII rilevate rientrino tra quelle dichiarate per l’attività di trattamento associata;
2. **Asse della conservazione:** per ciascun documento contenente PII, il sistema deve confrontare la data del documento (o il suo ultimo aggiornamento) con il periodo di retention dichiarato nel ROPA per l’attività di trattamento associata.

Criterio di verifica. Il requisito è soddisfatto se, dato un corpus documentale di test con anomalie note (PII non dichiarate e documenti oltre retention), il sistema le identifica correttamente in entrambi gli assi.

2.2.2 Riferimento al registro delle attività di trattamento come fonte normativa

Il sistema deve trattare il/i ROPA forniti in input come riferimento normativo autoritativo per ogni valutazione di conformità. Il sistema non deve generare, inferire o modificare autonomamente le attività di trattamento: il ROPA rappresenta la *ground truth* dichiarativa dell'organizzazione [11].

Razionale. L'art. 30 GDPR [2] e l'art. 12 nLPD [3] impongono al titolare del trattamento la tenuta di un registro che documenti, tra l'altro, le categorie di interessati, le categorie di dati personali trattati, le finalità, la durata di conservazione e le categorie di destinatari. Il sistema si fonda su questo documento come base di confronto, senza sindacarne la correttezza né sostituirsi alla responsabilità del titolare nella sua compilazione.

Qualora il ROPA fornito risulti incompleto o incoerente (es. attività di trattamento prive di periodo di conservazione), il sistema deve:

- segnalare le carenze nel report, classificandole come anomalie del ROPA stesso;
- proseguire l'analisi limitatamente ai campi disponibili, senza interrompere l'elaborazione.

Criterio di verifica. Il requisito è soddisfatto se il sistema, ricevendo un ROPA con campi mancanti, produce le segnalazioni appropriate senza errori di esecuzione e senza alterare i dati del ROPA.

2.2.3 Copertura delle categorie di dati degni di particolare attenzione

Il sistema deve essere in grado di riconoscere le categorie di dati personali degni di particolare protezione secondo l'art. 5 lett. c nLPD [3] e le categorie particolari di dati personali secondo l'art. 9(1) GDPR [2], segnalandone la presenza con priorità elevata nel report delle anomalie, in considerazione del regime di consenso espresso richiesto (art. 6 cpv. 7 lett. a nLPD [3]; art. 9(2) GDPR [2]).

2.2.4 Posizionamento rispetto all'art. 32(1)(d) GDPR

Il sistema, nella sua esecuzione periodica, deve costituire un'implementazione tecnica della procedura di *test, verifica e valutazione regolare dell'efficacia delle misure tecniche e organizzative* prevista dall'art. 32(1)(d) GDPR [2] e, analogamente, dall'art. 8 nLPD [3].

Razionale. L'art. 32(1)(d) GDPR [2] qualifica come misura di sicurezza obbligatoria una procedura per testare e valutare regolarmente l'efficacia delle misure adottate. Un sistema

che rileva periodicamente i dati personali nei documenti aziendali e ne verifica la coerenza con il registro dei trattamenti implementa, di fatto, questa procedura con riferimento alla conformità documentale.

Il perimetro del sistema è tuttavia limitato alla verifica di conformità documentale. Restano esplicitamente esclusi:

- funzioni di anonimizzazione o pseudonimizzazione dei dati;
- gestione degli incidenti di sicurezza (*data breach*);
- valutazione d'impatto sulla protezione dei dati (DPIA).

Criterio di verifica. Il requisito è soddisfatto se il sistema è in grado di essere eseguito periodicamente (es. tramite schedulazione automatica) producendo report comparabili tra esecuzioni successive.

2.2.5 Esclusione delle basi giuridiche dal perimetro di analisi automatica

Il sistema non deve determinare autonomamente la base giuridica applicabile a un trattamento (art. 6 GDPR [2]; art. 30-31 nLPD [3]). Tale valutazione resta di competenza esclusiva del DPO o del consulente giuridico dell'organizzazione.

Razionale. La determinazione della base giuridica richiede una valutazione contestuale di natura giuridica (consenso, interesse legittimo, obbligo legale, ecc.) che non è riducibile a un'analisi automatica del contenuto documentale. Come evidenziato nella sezione 2.2.1, il GDPR e la nLPD adottano inoltre approcci strutturalmente diversi alla liceità del trattamento, rendendo impossibile un'automazione generalizzata senza introdurre rischi di errore significativi.

Il sistema si limita pertanto a:

- segnalare la presenza di PII non riconducibili ad alcuna attività di trattamento dichiarata nel ROPA ("PII orfane");
- fornire al DPO gli elementi contestuali (documento, posizione, categoria di PII) necessari per una valutazione umana.

Criterio di verifica. Il requisito è soddisfatto se il sistema, in nessun caso, associa autonomamente una base giuridica a un trattamento e se le PII non associabili ad alcuna attività di trattamento vengono correttamente segnalate come "orfane".

2.3 Requisiti operativi

2.3.1 Elaborazione locale dei dati

Ogni dato analizzato dal sistema deve rimanere nell'infrastruttura in cui si trova inizialmente. Nessun contenuto documentale, testo estratto, PII rilevata o dato intermedio deve essere trasmesso a servizi esterni, API cloud o endpoint di inferenza remoti, in nessuna fase dell'elaborazione.

Razionale. I documenti analizzati possono contenere categorie di dati degni di particolare protezione ai sensi dell'art. 5 lett. c nLPD [3] (dati sulla salute, opinioni politiche, appartenenza religiosa, procedimenti penali, misure di assistenza sociale) e categorie particolari ai sensi dell'art. 9 GDPR [2]. La trasmissione di tali dati a servizi terzi introdurrebbe un trattamento ulteriore che richiederebbe una base giuridica propria, un'analisi dei rischi specifica e garanzie contrattuali (art. 28 GDPR [2]), vanificando la finalità stessa del sistema.

Il vincolo si applica a tutte le componenti del sistema, inclusi:

- i modelli di AI e NLP (cfr. 2.3.2);
- i moduli di estrazione del testo (cfr. 2.1.2);
- il registro interno delle PII (cfr. 2.1.10);
- la generazione e la presentazione del report di conformità (cfr. 2.1.9).

Criterio di verifica. Il requisito è soddisfatto se, durante l'esecuzione del sistema, non viene effettuata alcuna connessione di rete in uscita per la trasmissione di dati documentali o derivati. La verifica può essere condotta tramite analisi del traffico di rete o ispezione del codice sorgente.

2.3.2 Vincolo sui modelli AI impiegabili

I modelli di intelligenza artificiale impiegati dal sistema devono essere eseguibili interamente in locale, senza richiedere la trasmissione dei dati analizzati a servizi di inferenza esterni. Tale vincolo discende direttamente dal requisito di elaborazione locale (cfr. 2.3.1) e dalla natura potenzialmente sensibile dei dati trattati.

La scelta dei modelli deve bilanciare accuratezza e sostenibilità computazionale:

- per il rilevamento sistematico delle PII sull'intero corpus documentale, il sistema deve privilegiare approcci computazionalmente leggeri (rule-based, NER), eseguibili su hardware aziendale standard [12, 13];

- per attività selettive che richiedono comprensione semantica profonda (cfr. 2.1.13), il sistema può impiegare modelli generativi di dimensioni compatibili con l'esecuzione locale [10], limitandone l'applicazione a sottoinsiemi mirati del corpus.

La giustificazione della scelta architetturale dei modelli deve essere documentata attraverso studi comparativi che confrontino i risultati ottenuti con il consumo di risorse sostenuto (cfr. Capitolo 10).

2.3.3 Facilità di installazione e aggiornamento

Il sistema deve essere installabile e operativo in qualsiasi infrastruttura aziendale con un numero minimo di passaggi di configurazione, senza richiedere personalizzazioni specifiche per l'ambiente di destinazione.

Razionale. Il sistema è destinato a organizzazioni con livelli di maturità IT eterogenei. Un processo di installazione complesso o dipendente da configurazioni ad-hoc ridurrebbe l'adozione e renderebbe difficoltosa la manutenzione nel tempo.

In particolare, il sistema deve soddisfare i seguenti vincoli:

- l'installazione deve avvenire tramite un meccanismo standardizzato e riproducibile (es. container Docker, pacchetto con dipendenze integrate);
- la configurazione iniziale deve limitarsi alla specifica dei percorsi di input (documenti e ROPA) e delle preferenze di output;
- l'aggiornamento del sistema (nuove versioni, nuovi detector, aggiornamento dei modelli) deve poter avvenire senza perdita dei dati del registro interno (cfr. 2.1.10) e senza richiedere una reinstallazione completa;
- in caso di malfunzionamento, il sistema deve poter essere riportato a una configurazione funzionante nota senza interventi specialistici.

Criterio di verifica. Il requisito è soddisfatto se l'installazione del sistema su un ambiente pulito può essere completata seguendo la documentazione fornita, senza interventi manuali non documentati.

2.3.4 Estensibilità delle categorie e dei detector

Il sistema deve adottare un'architettura modulare che consenta l'aggiunta, la rimozione e la modifica delle categorie di PII rilevabili e dei relativi detector senza richiedere modifiche al codice sorgente del sistema.

Razionale. Le categorie di dati personali rilevanti variano in funzione del framework normativo applicabile (nLPD, GDPR, CCPA), del settore di attività dell'organizzazione e delle specifiche

attività di trattamento dichiarate nel ROPA. Un sistema non estensibile costringerebbe l'organizzazione ad attendere un aggiornamento del prodotto per coprire categorie specifiche del proprio contesto.

L'estensibilità deve essere garantita su due livelli:

- **Categorie di PII:** l'operatore deve poter definire nuove categorie di dati personali (es. numero di matricola aziendale, codice paziente) specificandone il nome, la descrizione e le regole di rilevamento associate;
- **Detector:** il sistema deve supportare l'aggiunta di nuovi moduli di rilevamento (regole regex, modelli NER specializzati, dizionari) tramite un'interfaccia di configurazione dichiarativa (es. file YAML o JSON), senza richiedere competenze di sviluppo software.

Criterio di verifica. Il requisito è soddisfatto se un operatore non sviluppatore è in grado di aggiungere una nuova categoria di PII con il relativo detector, seguendo la documentazione fornita, e il sistema la rileva correttamente al successivo ciclo di analisi.

2.3.5 Minimizzazione dei dati trattati dal sistema stesso

Il sistema, nel corso della propria esecuzione, deve rispettare il principio di minimizzazione dei dati (art. 5(1)(c) GDPR [2]; art. 6 cpv. 2 nLPD [3]), limitando la quantità di informazioni personali che tratta e conserva internamente a quanto strettamente necessario per produrre il report di conformità.

In particolare:

- il sistema non deve duplicare integralmente i documenti analizzati; il contenuto estratto deve essere mantenuto in memoria per la sola durata dell'elaborazione;
- le PII rilevate devono essere riportate nel registro interno (cfr. 2.1.10) e nel report finale attraverso riferimenti posizionali e non tramite la riproduzione integrale del dato personale, salvo i casi in cui ciò sia necessario per la comprensibilità dell'anomalia;
- i dati intermedi (testo estratto, strutture temporanee) devono essere eliminati al termine dell'esecuzione.

2.3.6 Operatività senza addestramento iniziale (zero-training)

Il sistema deve essere operativo immediatamente dopo l'installazione, senza richiedere una fase di addestramento su dati specifici dell'organizzazione. Il rilevamento delle PII deve basarsi su conoscenze preconfigurate (modelli pre-addestrati, regole predefinite, dizionari standard) sufficienti a garantire un livello di detection accettabile per le categorie canoniche della nLPD e del GDPR.

Il sistema deve tuttavia supportare meccanismi di personalizzazione facoltativa, attivabili dopo il primo impiego:

- dizionari specifici per il framework normativo di riferimento (es. categorie nLPD, GDPR, CCPA);
- specializzazioni basate su documenti rappresentativi dell'organizzazione, per migliorare la precisione del rilevamento su terminologia e formati specifici del dominio aziendale.

2.4 Requisiti di qualità

I requisiti di qualità definiscono le proprietà non funzionali che il sistema deve soddisfare per essere considerato adeguato al contesto operativo previsto. Essi riguardano in particolare le prestazioni su volumi documentali realistici, l'affidabilità del rilevamento e la capacità di giustificare ogni anomalia segnalata.

2.4.1 Prestazioni su larga scala

Il sistema deve essere in grado di analizzare volumi documentali rappresentativi di un'organizzazione di medie dimensioni (nell'ordine delle migliaia di documenti) in tempi compatibili con un'esecuzione periodica (es. settimanale o mensile).

L'architettura deve prevedere meccanismi di parallelizzazione dell'elaborazione e, ove necessario, di analisi incrementale, in modo da evitare la rielaborazione integrale del corpus documentale a ogni esecuzione. Il tempo di elaborazione deve scalare in modo al più lineare rispetto al numero di documenti analizzati.

2.4.2 Precisione del rilevamento

Il sistema deve privilegiare un elevato valore di *recall* (sensibilità), al fine di minimizzare il rischio che dati personali effettivamente presenti nei documenti non vengano segnalati (falsi negativi). Un falso negativo in questo contesto equivale a una non-conformità non rilevata, con potenziali conseguenze sanzionatorie per l'organizzazione.

Al contempo, il tasso di falsi positivi deve essere mantenuto a un livello tale da non compromettere l'usabilità del report per il DPO: un numero eccessivo di segnalazioni errate riduce la fiducia nel sistema e ne vanifica l'utilità operativa.

Le soglie di *precision* e *recall* accettabili per ciascuna categoria di PII sono definite in fase di validazione sperimentale (cfr. Capitolo 7).

2.4.3 Tracciabilità delle anomalie

Ogni anomalia segnalata nel report deve essere completamente tracciabile fino alla sua origine. In particolare, il sistema deve associare a ciascuna anomalia:

- il documento sorgente (percorso, nome file, eventuale hash);
- la posizione specifica all'interno del documento (pagina, paragrafo, riga o cella, ove applicabile);
- la regola o il modello che ha generato il rilevamento (es. identificatore del detector, categoria di PII rilevata);
- l'attività di trattamento del ROPA a cui l'anomalia è stata associata (o l'indicazione esplicita di assenza di associazione).

La tracciabilità è condizione necessaria per consentire al DPO di verificare ciascuna segnalazione e, ove necessario, di utilizzare il report come evidenza documentale nell'ambito delle procedure di *accountability* richieste dal GDPR (art. 5(2)) e dalla nLPD.

2.5 Requisiti di interfaccia

2.5.1 Interfaccia web come modalità primaria

Il sistema deve esporre un'interfaccia web come modalità primaria di interazione, servita dal sistema stesso su una porta di rete e raggiungibile da browser senza installazione lato client. L'interfaccia deve consentire almeno le seguenti operazioni:

- avviare l'analisi di una cartella, indicandone il percorso o selezionandola dal browser, con anteprima dei file che verranno analizzati e visualizzazione dello stato di avanzamento;
- fornire il/i ROPA in input e revisionare le categorie di dati riconosciute (cfr. 2.1.6);
- consultare il registro delle PII rilevate e il verdetto di conformità per ciascun documento (cfr. 2.1.10);
- gestire l'associazione tra documenti e attività di trattamento (cfr. 2.1.7).

Razionale. Il destinatario del sistema è il DPO, figura non necessariamente tecnica, e il sistema è distribuito come container (cfr. 2.3.3): un'interfaccia servita dal container stesso è operabile senza accesso alla shell dell'host e senza conoscenza dei comandi, a differenza di una CLI.

Criterio di verifica. Il requisito è soddisfatto se, a partire dal solo avvio del container, un utente è in grado di importare un ROPA, avviare una scansione e consultare il verdetto di conformità senza ricorrere alla riga di comando.

2.5.2 Interfaccia a riga di comando (CLI)

Il sistema deve esporre, come modalità secondaria, interfacce a riga di comando destinate all'automazione e alla diagnosi. La CLI è il punto di aggancio per l'esecuzione periodica non presidiata (cfr. 2.2.4) e per l'ispezione dei singoli stadi dell'elaborazione. Deve consentire almeno:

- l'ingestione di un ROPA e la mappatura delle sue categorie;
- la scansione di un singolo documento o di un albero di cartelle, con aggiornamento del registro interno;
- la verifica di conformità e la stampa del relativo verdetto.

Non è richiesto che tali operazioni siano esposte da un unico comando orchestratore: è ammessa una CLI per stadio dell'elaborazione, purché ciascuna sia componibile in uno script di schedulazione. L'interfaccia deve produrre messaggi di log progressivi che consentano di monitorare l'avanzamento e di diagnosticare gli errori.

2.5.3 Fruizione del report

Il report di conformità (cfr. 2.1.9) deve essere fruibile in due modi complementari:

- **consultazione interattiva:** l'interfaccia web (cfr. 2.5.1) deve presentare il report al DPO in forma navigabile — elenco dei documenti analizzati, dettaglio delle PII rilevate e verdetto per documento — senza richiedere un passo di esportazione né strumenti esterni;
- **lettura meccanica:** il contenuto del report deve poggiare sul registro interno persistente (cfr. 2.1.10), in un formato strutturato e interrogabile, così da restare accessibile a strumenti di governance, risk e compliance (GRC) o di analisi già in uso nell'organizzazione.

Razionale. Il report non è un artefatto separato ma una *vista* sui dati che il sistema già mantiene: il registro è la fonte di verità, l'interfaccia ne è la presentazione. Un file di report generato a fine analisi sarebbe invece una fotografia che invecchia — disallineata alla prima ri-scansione — e duplicherebbe in un secondo formato informazioni già persistite, con l'onere di mantenerli coerenti. La consultazione dal browser garantisce inoltre che il DPO veda sempre lo stato corrente del corpus documentale, e l'assenza di file di report esportati è coerente con il vincolo di minimizzazione (cfr. 2.3.5): non si moltiplicano le copie del quadro delle PII dell'organizzazione.

Ambito futuro. L'esportazione del report in file autonomi — strutturati (es. JSON, CSV) per l'integrazione con sistemi GRC, o impaginati (es. PDF, HTML) per la presentazione in sede

di audit — non rientra nel perimetro del prototipo: è una resa aggiuntiva sopra i medesimi dati, che non aggiunge capacità di analisi né di verifica.

Criterio di verifica. Il requisito è soddisfatto se, al termine di un'analisi di test, il DPO può consultare dal browser l'elenco dei documenti analizzati e il verdetto di ciascuno, e se il medesimo contenuto informativo è ricavabile dal registro interno mediante interrogazione diretta, senza passare dall'interfaccia.

Capitolo 3

Stato dell'arte

3.1 Background tecnologico

Questa sezione è specificamente indicata per inserire definizioni e spiegazioni su argomenti discussi successivamente.

3.1.1 Spiegazione supervised vs unsupervised

Come spiegato in uno dei testi cardine del machine learning, redatto da Goodfellow et al. [14], queste due metodologie di apprendimento si distinguono in base al tipo di *esperienza* che l'algoritmo acquisisce a partire dai dati durante l'addestramento. Sono definite come segue:

- per **Supervised learning** si intende quel paradigma di apprendimento automatico in cui l'algoritmo viene addestrato su un insieme di dati in cui ogni esempio, oltre alle sue caratteristiche, è associato a un'etichetta o a un valore target noto a priori. Il modello apprende quindi una funzione che mappa le caratteristiche di input all'etichetta corrispondente, sulla base di esempi già classificati o annotati. Un caso tipico è quello di un classificatore addestrato a riconoscere la specie di appartenenza di un fiore a partire da misurazioni morfologiche etichettate.
- per **Unsupervised learning** si intende quel paradigma in cui l'algoritmo opera su un insieme di dati privo di etichette, con l'obiettivo di individuare autonomamente strutture, regolarità o proprietà latenti presenti nei dati stessi. Nel contesto del deep learning, questo si traduce spesso nel tentativo di apprendere — in modo esplicito (ad esempio tramite stima di densità) o implicito (ad esempio per compiti di generazione o denoising) — la distribuzione di probabilità sottostante ai dati. Altri algoritmi non supervisionati svolgono invece compiti diversi, come il clustering, che consiste nel raggruppare gli esempi del dataset in base alla loro similarità.

Nel contesto specifico della NER, l'approccio supervisionato mostra miglioramenti quantificabili rispetto alle alternative zero-shot. La curva di apprendimento dei modelli Transformer pre-addestrati è ripida: con soli ~ 100 esempi annotati, l' $F1$ -score aumenta di circa 5.6 punti rispetto alla baseline zero-shot, mentre con 500–1000 esempi si raggiunge tipicamente un $F1$ del 90–95%. I rendimenti diventano decrescenti oltre i 2000 esempi, dove il modello raggiunge la saturazione dominio-specifica ($F1 \approx 95$ –99%). Fine-tunare su un dominio specifico comporta miglioramenti particolarmente netti per le categorie inizialmente mal coperte: le categorie con riconoscimento zero-shot scarso possono mostrare incrementi $F1$ fino al 57–80%.

3.1.2 Pre-training vs fine-tuning

Il paradigma pre-training/fine-tuning è stato reso centrale nell'NLP moderno dal lavoro di Devlin [15], che lo applicano al modello BERT (Bidirectional Encoded Representations from Transformers). Le due fasi si articolano come segue:

- per **pre-training** si intende la fase iniziale di addestramento in cui il modello apprende rappresentazioni linguistiche generali a partire da grandi quantità di testo non etichettato, tramite obiettivi di tipo *self-supervised* (ad esempio indovinare delle parole mascherate casualmente all'interno di una frase). In questa fase il modello non è ancora specializzato su alcun compito specifico: l'obiettivo è costruire una rappresentazione del linguaggio sufficientemente generale per poter essere riutilizzata per molteplici task.
- per **fine-tuning** si intende la fase successiva, in cui i parametri del modello pre-addestrato vengono ulteriormente ottimizzati su un dataset etichettato e di dimensioni ridotte, specifico per il compito che si intende risolvere. A differenza del pre-training, che richiede risorse computazionali ingenti, il fine-tuning può avvenire con un costo computazionale contenuto, poiché parte da una rappresentazione linguistica già appresa e la adatta al task target, spesso aggiungendo un solo livello di output specifico per il compito.

3.1.3 Spiegazione zero-shot learning

Il concetto di zero-shot learning è stato formalizzato in modo sistematico da Xian et al. [16], che lo definiscono come quel paradigma di apprendimento in cui un modello viene valutato su classi (o, nel caso della Named Entity Recognition, su tipologie di entità) che non erano presenti nel suo insieme di addestramento.

A differenza dei paradigmi supervised e unsupervised descritti in 3.1.1, lo zero-shot learning non riguarda la presenza o assenza di etichette nei dati di training, bensì la capacità del modello di generalizzare a categorie mai osservate, sfruttando informazioni semantiche

ausiliarie (ad esempio descrizioni testuali delle classi target) anziché esempi annotati specifici per quelle categorie. Tale capacità si distingue dal *one-shot* e dal *few-shot learning*, in cui il modello dispone rispettivamente di uno o di un numero ridotto di esempi annotati per le classi target [17].

3.2 Definizione e inquadramento del problema: Dalla Data Anonymization alla Data Governance

da capire cosa mettere in questa sezione, per non essere ripetitivo con quella del contesto
Da definire il problema tecnico dell'astrazione. (pensiero grezzo) I documenti non strutturati soffrono del problema del "Domain-Silo" [18]: i sistemi di detection falliscono perché addestrati su dati specifici e non riescono a generalizzare su testi eterogenei. Il problema centrale non è "leggere" il documento, ma togliere le ambiguità dalle informazioni in base al contesto in cui si trovano. *RIFORMULARE PER UNIRE TUTTO*

3.3 Compliance checking automatizzato

Sezione contenente metodi per confrontare il ROPA con i dati effettivamente all'interno dei dati.[11] **MAGARI DA SPOSTARE**

3.4 Approcci tecnici di PII detection

Per rilevare le PII, esistono più metodi e strategie, tutte ampiamente discusse in studi accademici. Ognuna di queste ha dei punti di forza e di debolezza, infatti esiste una area di problemi dove ogni singola opzione diventa la migliore, riducendo o addirittura annullando gli effetti negativi. (Approfondimento al capitolo 10 [DA LINKARE NEL DOCUMENTO])

Le soluzioni in questione sono:

3.4.1 Recognition rule-based

Approccio più semplice e rule-based. La detection delle PII è solamente affidata a regole di pattern-matching e meccanismi simili. Come si può già intuire, questo approccio, per alcune categorie di dati sensibili, è particolarmente efficace: la massima performance si ottiene quando si trattano dati strettamente formattati in determinati modi (IBAN, AVS, codice fiscale etc...). Non appena i testi diventano liberi e non strutturati, allora l'accuratezza crolla. [18, Sezione 5.2] Questo approccio tuttavia, ha dei problemi seri che non possono essere trascurati: [da capire se mettere lista oppure lasciare testo] categorie nettamente più difficili da trovare (nomi propri per esempio, in quanto si baserebbero su regole facilmente mal-interpretabili e con un alto numero di falsi positivi (es. lettera maiuscola) [IDEA: libreria

per checkare se una parola sia nel dizionario italiano/inglese. SE NO, ALLORA NOME PROPRIO])

Vantaggio: più veloce in assoluto tra tutte le soluzioni.

3.4.2 Recognition con NLP - NER

I modelli di Natural Language Processing (NLP), tramite la Named Entity Recognition (NER), classificano le entità testuali analizzando la struttura logica della frase. Architetture basate su Transformers (es. BERT) superando i limiti delle regole, comprendendo il contesto e raggiungendo accuratezze fino al 99.55% [19].

Vantaggio: eccellente bilanciamento tra comprensione del testo ed efficienza computazionale.

Svantaggio: Rischio di calo prestazionale su dati aziendali particolarmente specifici senza fare fine-tuning su dataset simili.

3.4.3 Recognition con AI generativa

Ultima soluzione "pura" è l'utilizzo di LLM con AI generativa per l'individuazione di PII. Questo approccio, fortemente documentato e discusso, è senza dubbio il migliore per quanto riguarda solamente le performance. Infatti, seppur riesce ad avere un F1-score e una recall senza precedenti, ha un costo computazionale e di risorse senza precedenti, spesso ingestibile per la maggior parte delle macchine. Gli unici modi per rendere accettabili tempi di esecuzione è di down-scalare i modelli utilizzati, riducendo contesto e "intelligenza" dello stesso. Seppur di aiuto, più si scende con la dimensione e il peso dei modelli, più il vantaggio di usare gli LLM si assottiglia, fino ad azzerarsi rispetto all'uso dell'NLP.

Tuttavia, gli LLM hanno un vantaggio unico per quanto riguarda un aspetto fondamentale nel nostro problema: la presenza nativa del 'contesto'. Infatti gli LLM possiedono una comprensione semantica avanzata del linguaggio naturale. Tramite il paradigma Zero-Shot, identificano PII deducibili puramente dal contesto verbale (es. dedurre "minorenne" dalla frase "mio figlio, nato 3 anni fa") senza alcun addestramento. Inoltre, offrono interpretazioni del loro operato, agevolando gli audit di conformità [10].

3.4.4 Limitazioni nella soluzione

Come visto nella sezione precedente, la soluzione parzialmente migliore sembrerebbe essere la detection con NLP. Questo perché riesce ad avere una percentuale di matching particolarmente alta, senza aver bisogno della quantità ingestibilmente alta di risorse dell'AI generativa.

Tuttavia, questa soluzione diventa praticamente insignificante se teniamo in considerazione l'aspetto dello "**Zero training**". Infatti, da requisiti [LINK AI REQUISITI], il nostro sistema di

detection deve essere ready-to-use non appena installato, garantendo una buona percentuale di detection senza aver bisogno dei dati su cui sta lavorando. Le metodologie NLP - NER hanno alla base proprio la knowledge dei dati che si stanno analizzando.

3.4.5 Soluzioni ibride

Per soddisfare il vincolo estremamente stringente dello "Zero-Training", bilanciando l'elevata richiesta di risorse dell'AI generativa e la rigidità dei sistemi rule-based, lo **standard industriale** prevede un'architettura ibrida (es. Microsoft Presidio) [20]

3.4.6 Confronto empirico degli approcci

Le sezioni precedenti presentano ciascun approccio in termini qualitativi. Per motivare la scelta architetturale in modo rigoroso, è necessario un confronto quantitativo basato su evidenze empiriche dalla letteratura recente. Le metriche considerate sono precisione (P), recall (R), F1-score, costo energetico di inference e necessità di dati annotati.

3.4.6.1 Prestazioni dei sistemi rule-based

Le espressioni regolari raggiungono alta precisione su pattern strutturati come email, numeri di telefono e indirizzi IP, specialmente se accompagnate da post-processing per eliminare falsi positivi sistematici (ad esempio ISBN confusi con numeri di telefono) [21]. Tuttavia, il limite strutturale è la *brittleness*: i detectors rule-based non considerano il contesto del testo circostante e falliscono sistematicamente su PII contestuali [22]. La letteratura di survey su NER conferma il pattern: per i metodi rule-based la precisione è tipicamente alta, mentre il recall tende a essere basso a causa di regole domain-specific; il processo di introduzione di nuove regole è costoso [23].

3.4.6.2 Prestazioni dei modelli NER supervisionati

Con corpus annotati adeguati, i modelli encoder fine-tuned raggiungono le prestazioni più elevate in assoluto tra tutti gli approcci considerati. In un benchmark comparativo su documenti clinici, BERT, ModernBERT e ClinicalBERT raggiungono tutti $P = 0.99$ e $R = 0.99$ ($CIRE \approx 0.99$), con throughput di 1528–2446 token/secondo e costo energetico di inference nettamente inferiore ai modelli generativi [24]. Un modello DeBERTa fine-tuned sul dataset *ai4privacy* raggiunge $F1 = 0.9757$ su un benchmark PII multi-dominio [25].

Su documenti finanziari non strutturati (audit report, fatture), una pipeline ibrida NLP rule-based + ML + NER custom raggiunge $P = 94.7\%$, $R = 89.4\%$, $F1 = 91.1\%$ su dataset sintetici, con il 93% di accuratezza su documenti reali [26].

Il principale ostacolo non è il costo di inference (basso dopo il training) ma il costo di annotazione del corpus: le soluzioni supervisionate richiedono tipicamente un corpus annotato di

grandi dimensioni [27]. Questo costo è però da sostenersi una tantum (fine-tuning), non per ogni documento analizzato (distinzione fondamentale rispetto all'inference LLM, ricorrente).

3.4.6.3 Prestazioni dei modelli NER zero-shot

GLiNER (cfr. 3.1.3) usa un encoder bidirezionale (DeBERTa) con un meccanismo di span-extraction (individuare inizio e fine di una label, invece di processare parola per parola) prompt-style (inserire le etichette da estrarre come testo, senza averne di predefinite nell'architettura del modello): le label vengono fornite come input testuale e il modello identifica span corrispondenti nel documento senza retraining [28]. Test su dataset PII multi-domain mostrano GLiNER-base a circa l'81% di F1, superiore ad approcci spaCy o regex standard, con recall particolarmente elevato su ID numerici e date [29]. Il modello `gliner-pii-base-v1.0` raggiunge $F1 = 80.99\%$ ed è ottimizzato per applicazioni privacy-focused in cui i servizi NER cloud pongono rischi di conformità [30].

Tuttavia, la specificità di dominio rimane determinante anche per approcci zero-shot: su testo clinico, GLiNER scende a circa $F1 = 0.41$, perdendo oltre il 50% delle entità PHI rispetto ai modelli domain-specific [31]. Questo impone una valutazione empirica obbligatoria sul corpus aziendale target prima di escludere il fine-tuning.

3.4.6.4 Prestazioni dell'AI generativa

GPT-4o, LLaMA 3.3 70B e LLaMA 3.1 8B raggiungono tutti recall medio superiore a 0.90, ma la precisione rimane un problema irrisolto: anche il top performer GPT-4o raggiunge soltanto $P = 0.579$ su dataset educativi [32]. Su benchmark strutturati multi-task (PII-Bench), GPT-4o con metodi avanzati (Self-CoT, Self-Consistency) raggiunge $F1 \approx 0.76\text{--}0.77$, con Llama 3.1 e Qwen 2.5 in linea [18].

Il problema principale per il contesto di questa tesi è il **costo energetico ricorrente**. Luccioni, Jernite e Strubell mostrano una distinzione netta tra modelli discriminativi task-specific — con emissioni molto inferiori — e modelli generativi decoder-only: la generazione di testo richiede circa 0.047 kWh per 1000 query [**luccioniPowerHungryProcessing2024**]. Questo costo si moltiplica per ogni documento analizzato nel filesystem aziendale ed è ricorrente (ogni nuova analisi, ogni re-scan), mentre il fine-tuning di un encoder è un costo one-shot. A parità di F1, RoBERTa costa circa \$0.044 per 1000 esempi contro \$13.50 di GPT-4 — un rapporto di circa 300:1 [33].

3.4.6.5 Convergenza verso la pipeline ibrida

La convergenza della letteratura recente indica la pipeline ibrida come soluzione ottimale per il rilevamento PII in contesti di compliance. Il sistema RECAP (NeurIPS 2025) combina regex deterministiche con LLM context-aware in un framework modulare a tre fasi: (1) rilevamento regex, (2) classificazione NER, (3) risoluzione ambiguità via LLM. RECAP supera i modelli

NER fine-tuned dell'82% e i LLM zero-shot del 17% in weighted F1-score su 13 lingue low-resource [7].

Un sistema ibrido indipendente (CHPDA) documenta lo stesso principio: le regex eccellono su pattern chiaramente definiti ma mancano di flessibilità per gestire complessità contestuali; i modelli NER eccellono nell'identificare entità ambigue come nomi e indirizzi, difficili da catturare con regex, ma producono falsi positivi senza pattern guida. La combinazione risolve i limiti di entrambi [34].

La letteratura non esclude l'AI generativa in assoluto, ma ne circoscrive il ruolo: l'impiego su casi ambigui come supervisor di una pipeline ibrida è documentato come cost-effective [7]. L'esclusione riguarda l'inferencesistemica su tutto il corpus, non l'impiego selettivo su un sottoinsieme di casi irrisolti.

3.4.6.6 Matrice di confronto sintetica

La Tabella 3.1 riassume le evidenze empiriche discusse nelle sezioni precedenti.

Tabella 3.1: Confronto empirico degli approcci di PII detection basato su evidenze dalla letteratura.

Approccio	P	R	F1 tipico	Costo inf.	Corpus ann.	Energia training	Energia / 1k query
Regex / rule-based	Alta	Bassa-media	0.60-0.75	Minimo	No	Nessuno	Trascurabile
NER supervised	0.95-0.99	0.89-0.99	0.94-0.99	Basso	Sì (grande)	Basso (one-shot) ^a	~0.002 kWh ^b
NER zero-shot (GLiNER)	Media	Media	~0.81	Basso	No	Nessuno ^c	~0.002 kWh ^b
LLM generativo (GPT-4o)	0.58	>0.90	0.70-0.77	Altissimo	No	Altissimo ^d	~0.047 kWh ^b
Pipeline ibrida	Alta	Alta	+82% vs NER	Medio	Opzionale	Basso (NER component)	Basso-medio ^e

^a Il fine-tuning di un encoder richiede ~44 h GPU totali su A100; il costo è one-shot e non ricorrente [24]. ^b Dati misurati su A100: text classification ~0.002 kWh/1k inf., text generation ~0.047 kWh/1k inf. [luccioniPowerHungryProcessing2024].

^c GLiNER usa un encoder pre-addestrato senza fine-tuning task-specific [28]. ^d Il pre-training di LLM generativi richiede ordini di grandezza più energia dell'inference; la parità training/inference si raggiunge dopo 200-590 M query [luccioniPowerHungryProcessing2024]. ^e Regex + NER encoder con LLM solo su casi ambigui: il costo è dominato dalla componente NER [7].

3.5 Analisi del training

Storicamente, i modelli utilizzati per l'estrazione di informazioni si dividono in due macrocategorie: supervised e unsupervised (cfr. 3.1.1). Ognuno dei due approcci presenta vantaggi e svantaggi in termini di scalabilità. Numerosi studi, come quello di Al Qahtani [35], hanno dimostrato come i modelli supervisionati garantiscano performance superiori su pattern noti, ma richiedano onerose fasi di etichettatura manuale per adattarsi a nuovi scenari.

Tuttavia, nel contesto specifico della Named Entity Recognition (NER) moderna, il dibattito si è spostato su due paradigmi di training più evoluti per i modelli Transformer:

3.5.1 Pre-training e fine-tuning (Paradigma classico)

Il paradigma tradizionale prevede l'utilizzo di un modello linguistico pre-addestrato su vasti corpus testuali, il quale viene poi adattato (*fine-tuned*) tramite un dataset supervisionato

specifico per il dominio di interesse (es. dati finanziari o medici). Sebbene garantisca accuratezza elevata, questo approccio introduce due criticità:

- **Costi di annotazione:** l'adattamento richiede dataset etichettati massicci, che nel caso delle PII comporta spesso l'intervento manuale di esperti e solleva problematiche di privacy intrinseche alla manipolazione di dati reali.
- **Perdita di generalizzazione (Catastrophic Forgetting):** la specializzazione spinta su un singolo formato documentale può compromettere la capacità del modello di generalizzare su testi imprevisti.

Questi vincoli risultano particolarmente critici in ambienti operativi ristretti. Come evidenziato dallo studio di Zhang et al. [36] sul rilevamento di PII in architetture *smart edge computing*, l'esigenza di eseguire inferenza locale e decentralizzata richiede modelli estremamente robusti che non obblighino a continui cicli di riaddestramento per coprire nuove categorie di dati sensibili.

3.5.2 Span-matching e Zero-shot (Il paradigma GLiNER)

Per superare i colli di bottiglia del fine-tuning, la letteratura ha esplorato modelli capaci di identificare entità senza addestramento pregresso sulle etichette specifiche (Zero-shot). Architetture recenti, come GLiNER [28], sfruttano encoder bidirezionali leggeri per eseguire uno *span-matching*. In questa modalità, al modello viene passato sia il documento da analizzare sia le descrizioni testuali delle PII da trovare (es. "Numero di previdenza sociale"). Il modello calcola la similarità semantica tra le parole del testo e la descrizione dell'etichetta cercata, identificando il dato personale in tempo reale. Questo paradigma è perfettamente allineato alle necessità di flessibilità e privacy locale teorizzate da Zhang [36], eliminando il costo del fine-tuning e permettendo di estendere le categorie ricercate semplicemente aggiornando la lista di istruzioni testuali in input.

3.6 Sfide nell'elaborazione documentale

Sezione con strumenti considerati per trasformare file non strutturati in formati facilmente leggibili da regex o, più importante, AI (NLP, generativa etc...)

3.6.1 Docling

Spiegone di Docling

3.6.2 Altro...

Spiegno di altre tecnologie (che ora non ricordo)

3.7 Panoramica dei sistemi esistenti

L'analisi dello stato dell'arte evidenzia una profonda spaccatura tra gli strumenti open-source, spesso focalizzati su un singolo task tecnici (anonimizzare o estrazione brutta), e le suite commerciali, ricche di feature ma close-source e inaccessibili.

3.7.1 Sistemi open-source

- **Microsoft Presidio** è un framework open-source sviluppato da Microsoft, diventato uno standard industriale per l'identificazione e l'anonimizzazione. Offre un'architettura modulare basata su classi estensibili come *EntityRecognizer* (per i modelli di Machine Learning) e *PatternRecognizer* (per il pattern matching classico) [37]. Il motore di rilevamento (*Analyzer*) è disaccoppiato dal componente di anonimizzazione (*Anonymizer*), include riconoscitori pre-costruiti anche per il contesto italiano (ad esempio il codice fiscale, con validazione a checksum) e consente di collegare motori NER esterni.

Il suo **limite rispetto allo scopo del progetto** non è tecnologico ma di *finalità*: nasce per la rilevazione e l'anonimizzazione del dato, non per la governance e la verifica di conformità. Non implementa alcuna logica di registro dei trattamenti (ROPA), di gap analysis o di reportistica per il DPO, e non copre nativamente identificatori del contesto svizzero (ad esempio il numero AVS). Il suo motore di detection resta però riutilizzabile come componente di un sistema più ampio: questa possibilità è valutata esplicitamente nel Capitolo 6 (§6.4).

- **GLiNER**: architettura Zero-Shot basata su Transformers, in grado di identificare PII passandole dinamicamente come query, senza addestramento. Le sue iterazioni recenti (GLiNER2-PII) mostrano prestazioni eccellenti in contesti multilingua [28].

Limiti: è esclusivamente un "motore algoritmico". Prende stringhe in input ed estrae etichette. Non implementa il parsing di documenti (PDF/Word), non genera reportistiche strutturate e non possiede alcuna logica di compliance (ROPA). [38]

- **Scrubadub / Datafog**: strutture open-source modulari. Scrubadub utilizza un'architettura a *Detector* multipli (Regex, detector predefiniti, NLP esterni) [39].

Limiti: sono fortemente legati al test-masking e all'oscuramento. Spesso si affidano pesantemente al pattern matching deterministico (Regex) (cfr:3.4.1), una tecnologia ormai insufficiente da sola per gestire documenti destrutturati e ambigui.

3.7.2 Sistemi commerciali

L'analisi dei player di mercato è fondamentale per comprendere quali caratteristiche vengono ricercate dalle aziende per quanto riguarda la compliance. Le soluzioni commerciali offrono architetture olistiche, ma a costi e configurazioni strutturali spesso incompatibili con le Piccole e Medie imprese o con vincoli stretti di privacy locale.

- **Varonis:** piattaforma leader mondiale per la Data Security Posture Management (DSPM). Implementa un motore ibrido che vanta un'accuratezza del **98%** su dati strutturati e non strutturati (SharePoint, OneDrive, NAS) senza necessità di training sui dati del cliente [40]. Tra le feature distintive spicca la User and Entity Behavior Analytics (UEBA), che monitora pattern anomali di accesso ai file sensibili [41], unita a scansioni *incrementali* per abbattere i costi computazionali.

Limiti: Si tratta di un sistema massiccio focalizzato sul *controllo degli accessi* e la Data Security delle reti aziendali. I costi (circa 80.000 CHF/anno [42]) e la sua architettura chiusa lo rendono sovradimensionato e inadatto allo scope del progetto.

- : piattaforma di altissimo livello per la Data Privacy e Governance. Il suo punto di forza risiede nell'**Identity-aware discovery**: non si limita a trovare un dato sensibile, ma correla i record a specifiche persone fisiche (Data Subjects) incrociando diverse fonti [43]. Utilizza una tecnologia proprietaria (*Hyperscan*) per prioritizzare lo scanning tramite Machine Learning. Soprattutto, supporta la **gestione dei ROPA automatizzata**, mappando i dati scoperti contro la conformità normativa e gestendo le policy di retention (cancellando dati "redundant, obsolete, trivial" o semplicemente scaduti) [44].

Limiti: struttura enterprise dai costi preclusivi su preventivo. Per molte operazioni di classificazione si affida ad architetture Cloud o Ibride, che sollevano importanti problematiche di Data Sovereignty, scontrandosi con il vincolo del progetto di mantenere tutta l'inference all'interno dell'infrastruttura aziendale cliente.

- **PII Tools:** soluzione verticale sulla singola PII detection. Offre un approccio Machine Learning che, secondo l'azienda, riduce i falsi positivi delle classiche Regex assicurando maggiore comprensione del contesto [45]. Risulta particolarmente interessante per le PMI poiché distribuibile interamente **on-premise via Docker in pochi minuti** e garantisce una mappatura che indica la localizzazione spaziale esatta del dato all'interno del documento [46].

Limiti: pur essendo leggero, resta un software *black-box* a pagamento. Manda di trasparenza e non offre flessibilità per scambiare il modello AI sottostante qualora se ne presentasse uno migliore.

3.8 Sintesi del gap e posizionamento del progetto

Alla luce di questa panoramica, il **gap nel mercato** risulta evidente. Le organizzazioni, in particolare le PMI svizzere, si riscontrano con un panorama fortemente polarizzato:

1. **Soluzioni open-source:** Rilevano i dati in locale ma lo fanno con l'ottica prioritaria dell'*anonimizzazione* (Presidio), oppure offrono solo modelli potenti linguistici disconnessi da qualsiasi logica di governance e reportistica aziendale (GLiNER).

2. **Soluzioni Commerciali:** Dimostrano quali siano le reali necessità delle aziende, introducendo l'Identity-aware discovery, l'automazione dei ROPA, lo scanning incrementale e le dashboard per DPO. Tuttavia, lo fanno imponendo ecosistemi Black-box, costi esorbitanti e massicce dipendenze da Cloud/API di terze parti, con conseguenti problematiche di *Data Sovereignty*.

Posizionamento del progetto: il software proposto si inserisce al centro di questo divario. L'obiettivo è fornire uno strumento open-source ed eseguibile **esclusivamente in locale** (on-premise), rispondendo all'esigenza di confidenzialità totale. Abbandonando il paradigma dell'anonimizzazione dei tool, il ...

Sfruttando parser moderni (cfr.3.6) per affrontare dati strutturati e motori NLP Zero-Shot (cfr.3.1.3) per eliminare i costi di training descritti precedentemente. Il sistema non si limiterà a scoprire le PII, ma fungerà da ponte tra il dato reale e le policy aziendali (ROPA), fornendo un quadro clinico dell'aderenza normativa aziendale.

da inserire tabella riassuntiva (o paragrafo comparativo) che fa capire cosa "fanno gli altri" e "cosa manca".

Capitolo 4

Analisi dei rischi

4.1 Rischi di progetto

- 4.1.1 Disponibilità e qualità dei dataset di valutazione
- 4.1.2 Complessità dell'integrazione multi-approccio
- 4.1.3 Costo computazionale e tempi di elaborazione
- 4.1.4 Dipendenza da modelli e librerie di terze parti
- 4.1.5 Carenza di competenze specialistiche
- 4.1.6 Scope creep e requisiti in evoluzione

4.2 Rischi di prodotto

- 4.2.1 Accuratezza insufficiente nella detection delle PII
- 4.2.2 Falsi positivi e impatto sull'usabilità
- 4.2.3 Falsi negativi e mancata rilevazione di PII critiche
- 4.2.4 Generalizzazione cross-domain (domain-silo problem)
- 4.2.5 Supporto multilingue e multilocalizzazione
- 4.2.6 Scalabilità su filesystem di grandi dimensioni
- 4.2.7 Qualità e completezza dei ROPA in input
- 4.2.8 Conformità normativa e aggiornamento dei framework legali
- 4.2.9 Sicurezza e protezione dei dati durante l'analisi

4.3 Valutazione dei rischi

- 4.3.1 Criteri di classificazione

4.3.2 **Matrice probabilità-impatto** livello medio e basso per i rischi di sicurezza, ma non strutturati aziendali

4.4 Strategie di mitigazione

Capitolo 5

Architettura del sistema

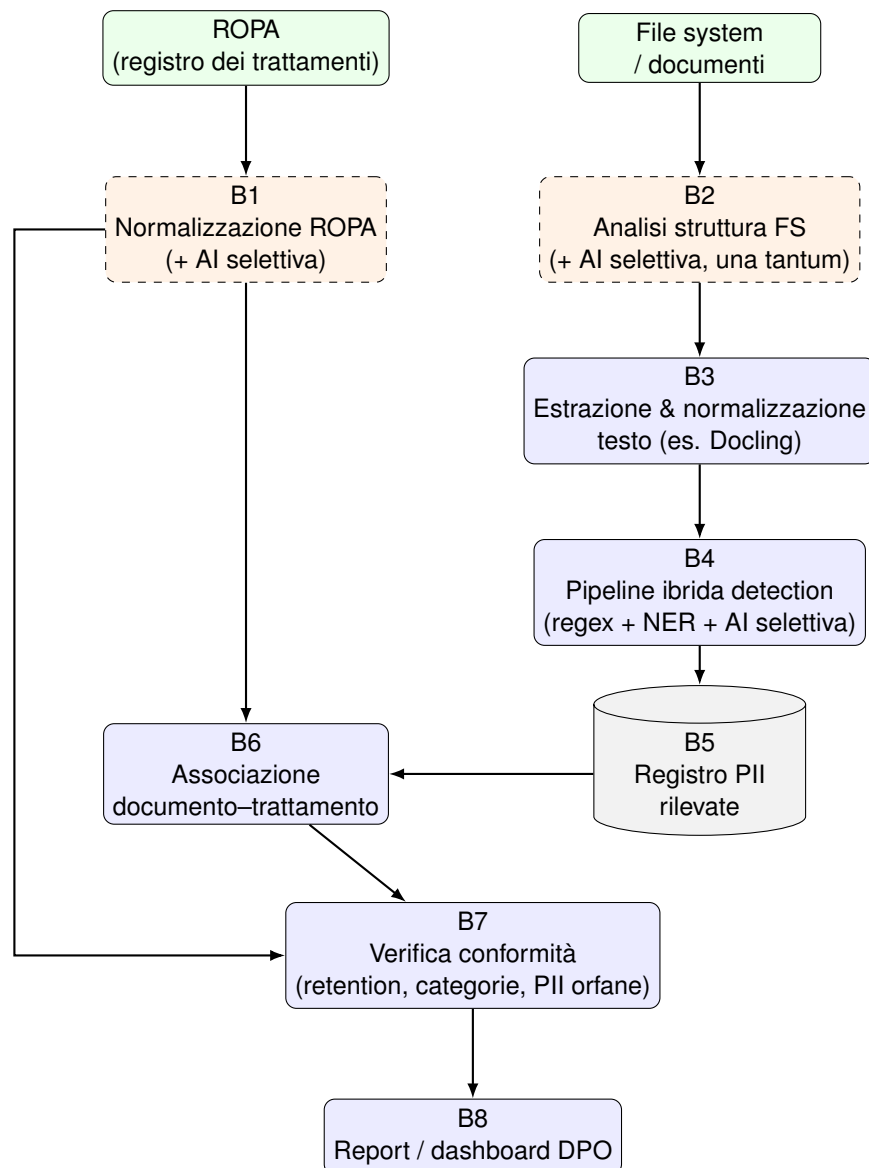


Figura 5.1: Architettura a macro-blocchi del sistema

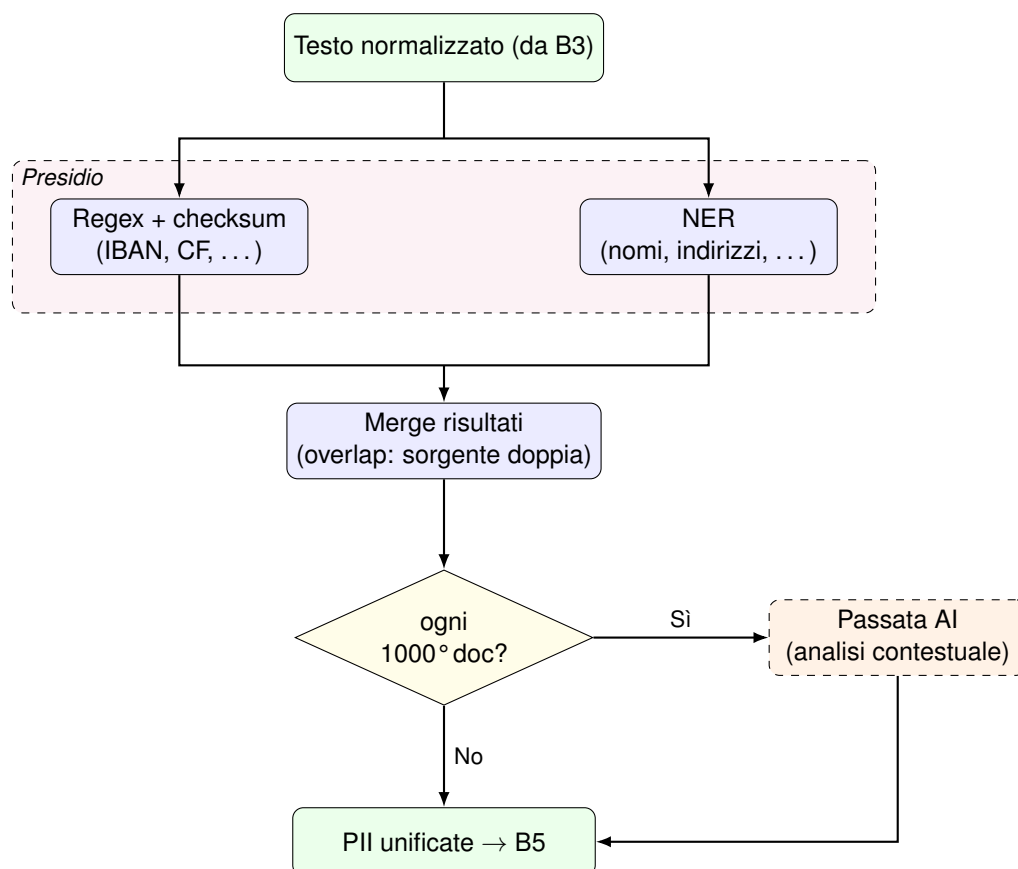


Figura 5.2: Dettaglio del blocco B4: pipeline ibrida di rilevamento PII. I detector regex e NER sono forniti da Presidio (AnalyzerEngine) dietro il contratto PIIDetector; il merge e la passata AI selettiva restano proprietari.

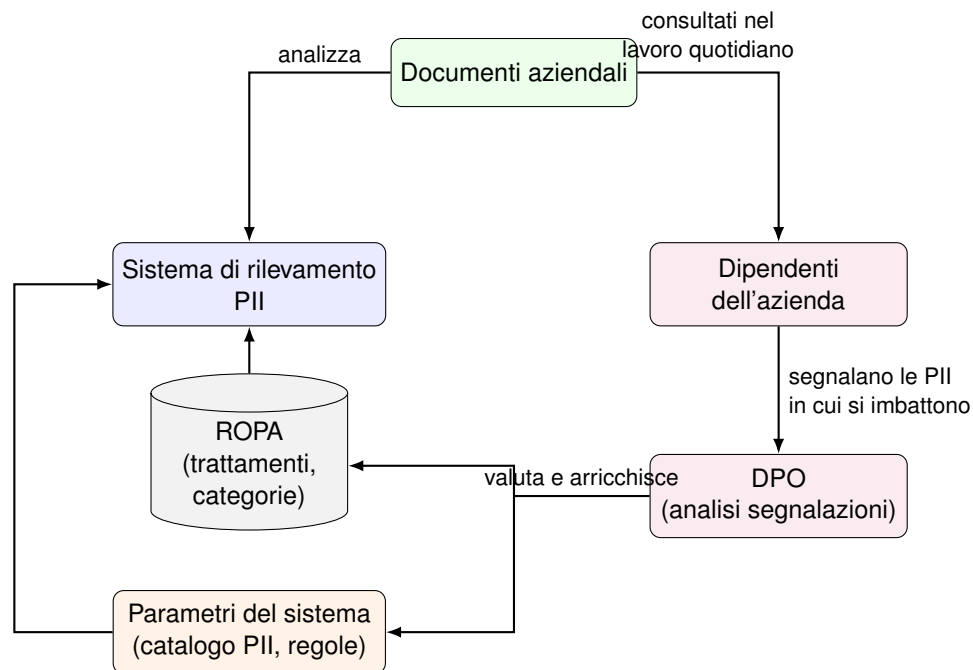


Figura 5.3: Meccanismo di *human-in-the-loop*: nel normale lavoro con i documenti aziendali i dipendenti segnalano al DPO le PII in cui si imbattono; il DPO analizza le segnalazioni e arricchisce il ROPA o i parametri del sistema, migliorando il rilevamento. Il processo di analisi resta trasparente ai dipendenti, per i quali i file sono semplici documenti.

5.1 Ingestione e modello dati del ROPA

Il blocco B1 della Figura 5.1 acquisisce il ROPA (*Record of Processing Activities*, il registro delle attività di trattamento) e lo trasforma in una rappresentazione interna. Non è una fase accessoria: il ROPA è il termine di paragone *atteso* rispetto al quale la verifica di conformità (B7) confronta le PII effettivamente rilevate nei documenti. La sua struttura dati condiziona quindi ciò che il sistema è in grado di verificare. Questa sezione motiva l'approccio scelto per modellarlo.

5.1.1 Strutturazione a monte o delega all'AI

L'ingestione del ROPA ammette due strategie agli estremi opposti. Nella prima la **struttura dati è definita a monte**: il sistema impone uno schema noto e l'ingestione lo popola in modo deterministico a partire da un ROPA già compilato. Nella seconda, **totalmente agentica**, i documenti del ROPA vengono forniti così come sono a un modello generativo, delegando a esso l'individuazione della struttura — l'approccio «si immettono i documenti e si osserva cosa emerge».

Si adotta la prima. Un registro dei trattamenti è un *artefatto di conformità*: la sua affidabilità e la sua auditabilità non possono dipendere da una strutturazione inferita, potenzialmente

allucinata e non riproducibile. Uno schema esplicito garantisce inoltre che il modello sia interrogabile in modo prevedibile dal blocco di confronto (B7) e mantenibile nel tempo. L'AI non viene esclusa, ma **circoscritta** a un ruolo di arricchimento su un singolo campo (§5.1.4); ciò è coerente con l'uso selettivo dell'AI già previsto per il blocco B1 nella Figura 5.1.

5.1.2 Il registro-tipo CNIL come schema di riferimento

Il contenuto obbligatorio del registro è fissato dall'art. 30 del Regolamento (UE) 2016/679 [**gdprArt30**] e, per il contesto svizzero, dall'art. 12 della LPD [**nlpdArt12**]; i due elenchi sono ampiamente sovrapponibili. In assenza di una pluralità di modelli operativi consolidati, il riferimento pratico più diffuso è il registro-tipo pubblicato dalla *Commission Nationale de l'Informatique et des Libertés* (CNIL), l'autorità di controllo francese, derivato direttamente dall'art. 30 [**cnilRegistreTraitement**, **cnilRegistreModeleBasique**]. Adottarlo come base evita di progettare uno schema dal principio e lega il modello ad una fonte ampiamente autorevole e condivisa.

Il modello CNIL è organizzato per *attività di trattamento*: una scheda per ciascun trattamento, con i campi seguenti.

- identificazione del trattamento (nome, date di creazione e aggiornamento, base giuridica);
- attori: titolare del trattamento, eventuali co-titolari, responsabile della protezione dei dati (DPO), responsabili del trattamento esterni (*processors*);
- finalità del trattamento;
- categorie di persone interessate;
- categorie di dati personali, con evidenziazione delle categorie particolari (dati sensibili);
- categorie di destinatari (interni, esterni, responsabili esterni);
- trasferimenti verso Paesi terzi (destinazione e garanzie);
- durate di conservazione;
- misure di sicurezza tecniche e organizzative.

5.1.3 Normalizzazione in un modello relazionale

La scheda CNIL è *denormalizzata* — un foglio piatto per trattamento — e ricca di campi che il sistema non ha motivo di modellare. Poiché la verifica di conformità (B7) confronta le categorie di dati *dichiarate* con quelle *rilevate*, il modello interno conserva soltanto ciò che alimenta quel confronto — l'identità del trattamento e le sue categorie di dati personali — e scarta deliberatamente gli altri campi della scheda (attori, categorie di interessati, destinatari,

trasferimenti verso Paesi terzi, misure di sicurezza). È una scelta **sottrattiva**: riduce la superficie da mantenere e da tenere allineata alla fonte, senza togliere nulla a ciò che il sistema deve poter verificare.

Ciò che resta viene normalizzato in una gerarchia a **tre livelli**, che ricalca l'organizzazione del registro CNIL:

- `ProcessingActivity` (livello 1) — l'identità del trattamento: un identificatore stabile, il nome e la finalità.
- `DeclaredMacroCategory` (livello 2) — un blocco di categorie di dato del registro (ad esempio «stato civile, identità»), con la relativa **durata di conservazione**. La conservazione vive a questo livello, non sul singolo dato: le categorie raccolte nel blocco la ereditano. Accanto al testo originale si conserva una durata in mesi *calcolabile a macchina* (`retention_months`), così la verifica «conservazione superata?» diventa un confronto e non una lettura di prosa.
- `DeclaredCategory` (livello 3) — la singola categoria dichiarata, il punto di aggancio al motore di rilevamento.

Un solo collegamento fa del ROPA il riferimento operativo del sistema, e non un semplice archivio. `DeclaredCategory` è messa in corrispondenza (relazione 1–N, tramite l'attributo `pii_types`) con il **catalogo delle categorie di PII** usato dal motore di rilevamento (blocco B4). Il ROPA dichiara così quali categorie un trattamento *dovrebbe* contenere; il blocco B4 rileva quelle *effettivamente* presenti nei documenti. Il confronto tra i due insiemi è ciò che alimenta la verifica di conformità (B7): da un lato le PII rilevate ma non dichiarate (*orfane*, cfr. §5.4.4), dall'altro le categorie dichiarate ma mai riscontrate. Una categoria che non si risolve in alcun `pii_type` (insieme vuoto) resta **esplicita** nel modello: è dichiarata ma non rilevabile dal motore, e come tale va segnalata al DPO invece di essere scambiata per conforme.

5.1.4 Ruolo circoscritto dell'AI: normalizzazione, non strutturazione

I livelli 1 e 2 vengono popolati in modo **deterministico**: poiché un ROPA compilato secondo il modello CNIL ha una struttura nota, leggere l'identità del trattamento, i blocchi di categorie e le durate di conservazione è un'operazione di parsing, senza AI. L'unico punto che richiede interpretazione è il passaggio al **livello 3**, cioè il contenuto dei blocchi di categorie, che nei registri reali è spesso redatto in linguaggio naturale e in modo sbrigativo (ad esempio «dati anagrafici, coordinate bancarie, contatti»). Qui l'AI interviene **una tantum e a livello di singolo campo**, per *scindere* ed espandere quel testo libero nelle singole categorie e risolverle sulle voci canoniche del catalogo di PII (ad esempio «coordinate bancarie» → `iban`; «dati anagrafici» → `person_name`, `date_of_birth`, `italian_id`). L'AI opera **dentro un vocabolario chiuso e verificato**: ogni `pii_type` proposto è validato contro il catalogo e gli identificatori sconosciuti vengono scartati, così il modello non può essere inquinato

da un'allucinazione. La mappatura è marcata come *proposta* (`MappingState.PROPOSED`) e sottoposta a **conferma del DPO** (*human-in-the-loop*) prima di essere fissata (`CONFIRMED`). Questo confina l'AI a un compito circoscritto, verificabile e reversibile, coerente con la strategia di uso selettivo dell'AI adottata altrove nel sistema, senza cedere alla strutturazione agentica dell'intero documento.

5.1.5 Punti aperti di progettazione

Sul modello del ROPA, a differenza di quello della persistenza, alcune decisioni restano tuttora aperte e si riportano qui come lavoro in corso.

5.1.5.0.1 Allineamento nLPD–GDPR. Il modello parte dai campi CNIL (impianto GDPR); vanno annotate le differenze rilevanti dell'art. 12 LPD, in particolare le soglie e le eccezioni previste per le piccole e medie imprese, che incidono su quali trattamenti debbano comparire nel registro.

5.1.5.0.2 ROPA non strutturato. Il modello assume un registro già compilato in forma tabellare. Il caso di un ROPA fornito come testo libero (non nel formato CNIL) richiederebbe un passo di strutturazione più invasivo: va deciso se supportarlo e in che misura, restando entro il vincolo di non delegare all'AI la strutturazione complessiva.

5.2 Livello di estrazione del testo

Il blocco B3 trasforma un documento nel suo formato originale (PDF, Word, testo semplice) nella sequenza di testo normalizzato su cui opera il rilevamento. È il confine che disaccoppia i due mondi: a monte i formati eterogenei dei documenti aziendali, a valle un unico contratto testuale — il *documento normalizzato*, costituito dall'identificatore del documento e dal suo testo — che B4 riceve senza dover conoscere il formato di provenienza (cfr. l'ingresso «Testo normalizzato» nella Figura 5.2).

5.2.0.0.1 Uno scope a stadi. L'estrazione affronta per prima la classe di documenti *born-digital*, quelli in cui il testo è già rappresentato come tale (PDF nativi, file Word): qui la lettura è diretta e priva di rumore. Restano fuori, come estensione pianificata e non come limite dell'impianto, due casi più onerosi: i documenti *scansionati*, che richiedono un passo di riconoscimento ottico dei caratteri (OCR), e i *layout* complessi (tabelle, testo multicolonna), che richiedono una ricostruzione della struttura di pagina. Entrambi si innestano dietro lo stesso confine — sostituendo o arricchendo l'estrattore senza toccare il livello di rilevamento — e sono coperti da strumenti dedicati come Docling.

5.2.0.0.2 Estrazione e rilevamento come *concern* separati. Tenere l'estrazione dietro un confine netto ha una conseguenza metodologica: la qualità dell'estrazione e quella del rilevamento si possono misurare *separatamente*, attribuendo un eventuale errore all'uno o all'altro stadio. È una proprietà di cui la validazione del rilevamento (Cap. 7) si avvale per isolare il costo introdotto dal passaggio attraverso l'estrazione.

5.3 Livello di rilevamento ibrido delle PII

Il blocco B4 della Figura 5.1 è il motore di rilevamento vero e proprio: dato il testo già estratto e normalizzato dal blocco a monte, individua le PII presenti nel documento e le consegna, unificate, al registro (B5). È un livello *ibrido*, che combina due tecniche indipendenti e complementari; la sua architettura interna è dettagliata nella Figura 5.2.

5.3.1 Due sorgenti complementari dietro un contratto unico

Le due tecniche coprono classi di PII diverse. Le **espressioni regolari** (pattern e, in prospettiva, verifica di *checksum*) riconoscono gli *identificatori strutturati* — IBAN, codice fiscale, numero di carta, indirizzo IP — dove la forma è rigida e la precisione può essere massima. Il **riconoscimento di entità nominate** (NER) copre invece i dati meno strutturati e dipendenti dal contesto — nomi di persona, indirizzi — dove nessuna regola fissa è sufficiente. Entrambe sono esposte al resto del sistema attraverso un unico contratto, `PIIDetector` (nella forma `detect(testo) → lista di candidati`): l'orchestrazione dipende solo da questa forma, non dalla tecnologia sottostante, così una sorgente può essere sostituita senza toccare il resto del livello (cfr. §6.4).

5.3.2 Il merge: unificazione e grado di certezza

Le due sorgenti producono candidati in modo autonomo; il *merge* li unifica in un unico insieme di PII, decidendone il grado di certezza in base a *quante* e *quali* sorgenti concordano su una stessa porzione di testo. Confrontando gli span per sovrapposizione, ogni PII risultante ricade in uno di quattro casi — un vocabolario chiuso:

- **Fonte singola** — rilevata da una sola tecnica, senza sovrapposizioni. Non viene mai scartata (cfr. il principio *recall-first* più sotto), ma etichettata come meno certa.
- **Doppia conferma** — pattern e NER concordano sullo stesso tipo di PII su span sovrapposti. È il caso più solido: la confidenza viene rinforzata.
- **Conflitto** — gli span si sovrappongono ma le due sorgenti assegnano tipi diversi. Il merge **non arbitra**: conserva entrambe le ipotesi e demanda la risoluzione al blocco a valle, che dispone di più contesto.

- **Scoperta dall'AI** — individuata dalla passata generativa selettiva (campionata), assente dalle altre sorgenti.

5.3.2.0.1 Principio recall-first. Nessuna sorgente scarta di propria iniziativa un candidato debole: al più lo segnala con confidenza bassa. La ragione è la finalità di conformità del sistema: un falso positivo costa al DPO una verifica in più, mentre una PII *mancata* è un rischio che resta invisibile. Il merge eredita questo principio — non elimina, ma unifica e classifica — lasciando ogni eventuale scarto a una decisione umana o a un blocco a valle.

5.3.2.0.2 Un solo modello dato. La PII unificata prodotta dal merge non richiede un modello dati intermedio: la sua *certezza* è già descritta da tre attributi che essa porta con sé — la confidenza (quanto è forte il rilevamento), il livello di conferma appena descritto, e l'elenco delle *provenienze* (quali e quanti detector concordano, ciascuno con la propria confidenza). Mantenere questi campi sul singolo oggetto, invece di duplicarli in strutture parallele, conserva un'unica fonte di verità per la PII rilevata e ne preserva la tracciabilità verso il DPO.

5.4 Persistenza del registro delle PII

Il blocco B5 della Figura 5.1 — il registro delle PII rilevate — non è un semplice contenitore dell'ultimo stato osservato, ma l'elemento su cui poggia la capacità del sistema di ragionare sull'evoluzione dei dati personali nel tempo. La scelta del modello di persistenza che lo sostiene non è quindi un dettaglio realizzativo, ma una decisione architetturale che condiziona quali domande il sistema è in grado di rispondere. Questa sezione formalizza il requisito che la persistenza deve soddisfare, confronta gli approcci valutati e motiva quello adottato.

Un vincolo di progetto attraversa tutta la discussione: **il registro non conserva i valori effettivi delle PII**, ma solo riferimenti a esse (tipo, posizione, documento di appartenenza). È una scelta di minimizzazione del dato — un registro di conformità non deve a sua volta trasformarsi in un archivio di dati personali — e ha una conseguenza diretta sul significato che possono assumere gli eventi di «modifica» (cfr. §5.4.5).

5.4.1 Il requisito di tracciamento temporale

Le linee guida di progetto richiedono che il sistema mantenga una traccia tale per cui, se un utente in seguito modifica un file, sia possibile accorgersi che una determinata PII è stata cancellata o modificata rispetto a una rilevazione precedente. Il registro deve cioè poter rispondere non solo alla domanda «quali PII sono presenti ora?», ma anche «*che cosa è cambiato* rispetto alla scansione precedente?».

Questo è un requisito di **tracciamento temporale** e ha una conseguenza diretta sul modello dati: non basta uno stato corrente scollegato dal passato, serve un meccanismo che legghi

esplicitamente le rilevazioni successive dello stesso documento. È il criterio rispetto al quale vengono valutate le opzioni che seguono.

5.4.2 Approcci valutati

Il problema non si riduce a una scelta binaria tra snapshot ed event logger: esistono tre approcci distinti, con profili di complessità e garanzie molto diversi, riassunti nella Tabella 5.1.

Tabella 5.1: Approcci valutati per la persistenza del registro delle PII.

#	Approccio	Idea di fondo	Fonte di verità
A	Snapshot completi	Ogni scansione salva l'intero stato rilevato, indipendente dalle altre	Ultimo snapshot
B	Event sourcing puro	Si salvano solo gli eventi; lo stato corrente è <i>derivato</i> col replay	Il log degli eventi
C	Changelog ibrido (delta-based)	Stato corrente mutabile e log append-only secondario delle variazioni	Lo stato corrente (log complementare)

5.4.2.1 Opzione A — Snapshot completi

A ogni esecuzione della pipeline si salva un'istantanea completa di tutte le PII rilevate (`pII_type`, `document_path`, `position`, `associated_processing`, `scan_date`) autoconsistente e indipendente dalle altre. Il modello dati è minimale (una sola entità, nessuna logica di correlazione), non esiste rischio di derivare uno stato errato perché ogni snapshot è lo stato, e la realizzazione è immediata.

Il limite è però esattamente sul requisito del §5.4.1: non esiste **alcuna relazione esplicita tra scansioni consecutive**. Con due snapshot presi a distanza di ore, ricostruire «che cosa è cambiato» richiede un confronto (diff) calcolato a posteriori, riga per riga, senza alcuna struttura che leghi le due fotografie. A ciò si aggiunge la ridondanza dei dati (le PII invariate vengono riscritte a ogni scansione) e il fatto che il tracciamento temporale non è una proprietà del modello ma un'operazione applicativa da implementare comunque. L'opzione non è scartata perché errata, ma perché spostata interamente sul livello applicativo il lavoro di correlazione che il requisito richiede, senza offrire una struttura dati che lo supporti.

5.4.2.2 Opzione B — Event sourcing puro

Lo stato dell'applicazione non viene mai salvato direttamente: si registra ogni cambiamento come evento immutabile in un log append-only, e lo stato corrente si ottiene *rigiocando* (replay) l'intera sequenza di eventi. Nella formulazione originale di Fowler, il log diventa la fonte di verità principale e lo stato è puramente derivato [fowlerEventSourcing2005]. Il pattern offre per costruzione una storia completa e immutabile — si può ricostruire lo stato a qualsiasi istante passato — con auditabilità nativa, proprietà interessante per un sistema di conformità; qui il tracciamento temporale non è un'aggiunta, ma il modello stesso.

Su questo approccio la letteratura è però netta. Lo studio empirico di Overeem et al., basato su interviste con 25 ingegneri che hanno adottato il pattern in sistemi reali, identifica cinque sfide ricorrenti, tra cui una curva di apprendimento ripida e l'evoluzione dello schema (*schema*

evolution): non difficoltà percepite, ma *liabilities* documentate del pattern [47]. Inoltre, poiché lo stato corrente non è memorizzato ma va ricostruito, anche l'interrogazione più semplice («quali PII ci sono ora in questo documento?») richiede di rigiocare gli eventi; per rendere efficienti le letture, l'event sourcing viene quasi sempre affiancato al pattern CQRS (*Command Query Responsibility Segregation*), che introduce un *secondo* modello di lettura mantenuto in parallelo, cioè ulteriore complessità architetturale [microsoftAzureEventSourcing, awsEventSourcingPattern, 48]. A questo si sommano la crescita illimitata del log e la gestione di proiezioni, versioning degli eventi e consistenza eventuale. La stessa documentazione di riferimento raccomanda di adottare il pattern **solo** quando i benefici di auditabilità e ricostruzione storica giustificano la complessità introdotta, notando che per la maggior parte dei sistemi la gestione dati tradizionale è sufficiente [microsoftAzureEventSourcing].

5.4.2.3 Opzione C — Changelog ibrido delta-based

L'approccio adottato mantiene **due** strutture complementari:

1. uno **stato corrente mutabile** — la lista delle PII attualmente presenti, interrogabile direttamente senza alcuna ricostruzione;
2. un **log append-only delle variazioni** (*changelog*), *secondario*, che registra ogni delta tra una scansione e la successiva e lega esplicitamente ogni variazione alla scansione precedente dello stesso documento.

La differenza cruciale rispetto all'opzione B è che il log **non è l'unica fonte di verità**: lo stato corrente esiste già come dato di prima classe, quindi nessun replay è necessario per le operazioni quotidiane. Il changelog serve a rispondere alla domanda «che cosa è cambiato», non a ricostruire lo stato. L'approccio combina due concetti presenti in letteratura: il **dataset delta-driven** proprio del *Change Data Capture* (CDC), che cattura le variazioni tra uno stato e il successivo *senza* fare del log la fonte di verità esclusiva, mantenendo lo stato corrente come dato normale e interrogabile [kleppmannDesigningDataIntensive2017] !!!!!; e l'idea di **log immutabile append-only** come traccia di audit, mutuata dall'event sourcing ma applicata in forma non esclusiva.

Nota terminologica. Il CDC descritto da Kleppmann cattura i cambiamenti leggendo il *transaction log* di un database sorgente. Nel sistema qui proposto, invece, i cambiamenti vengono *rilevati* confrontando i risultati di scansioni successive di documenti esterni: non esiste un database sorgente di cui intercettare le transazioni. Il modello è dunque *ispirato* al paradigma delta-based del CDC, non un'implementazione di CDC in senso stretto.

5.4.3 Motivazione della scelta

La scelta dell'opzione C è il risultato di un confronto esplicito di *trade-off*, sintetizzato nella Tabella 5.2, e si articola in quattro punti:

1. **Risolve il difetto di A.** Il changelog lega ogni variazione alla scansione precedente dello stesso documento: le scansioni non sono più fotografie isolate e la domanda «che cosa è cambiato» ha una risposta strutturale, non un diff calcolato a posteriori.
2. **Evita il costo di B.** Lo stato corrente è memorizzato e interrogabile direttamente: niente replay, niente proiezioni, niente CQRS, niente consistenza eventuale da gestire. Le sfide di adozione documentate da Overeem et al. [47] non si applicano, perché non si sta adottando event sourcing puro.
3. **Conserva l'auditabilità dove serve.** Se in futuro servisse la ricostruzione storica completa, resta possibile rigiocando il changelog — ma è una *possibilità*, non un requisito per il funzionamento quotidiano: si ottiene gran parte del beneficio di B senza pagarne il prezzo in anticipo.
4. **È coerente con la minimizzazione del dato.** Registrando solo riferimenti e non valori, sia lo stato corrente sia il changelog restano leggeri e privi di PII effettive.

Tabella 5.2: Sintesi comparativa dei tre approcci di persistenza.

Criterio	A — Snapshot	B — Event sourcing	C — Changelog ibrido
Relazione esplicita tra scansioni	X	✓	✓
Interrogazione stato corrente	immediata	replay / CQRS	immediata
Complessità realizzativa	bassa	alta	medio-bassa
Auditabilità / storia completa	X	✓	parziale (estendibile)
Ridondanza dei dati	alta	nessuna	bassa
Coerenza con minimizzazione del dato	✓	✓	✓
Rischio di stato derivato errato	nullo	presente	nullo
Sfide di adozione documentate	—	5 [47]	non applicabili

5.4.4 Modello dati concettuale

Il modello che segue è *technology-agnostic*: la scelta della tecnologia di persistenza concreta è discussa nel Capitolo 6. Oltre alle entità già definite nel modello (Document, Scan, ProcessingActivity), la persistenza introduce due entità principali.

PIIInstance rappresenta lo stato corrente, mutabile: «questa PII, in questo documento, così com'è ora». Comprende un identificatore, il pii_type, il riferimento al Document, la current_position, un riferimento opzionale (0..1) al ProcessingActivity — assente se la PII è *orfana*, cioè non riconducibile ad alcun trattamento dichiarato — e il riferimento all'ultima Scan che l'ha osservata. È la tabella che il report per il DPO legge **direttamente**.

PIIChange è il log append-only, secondario, e rappresenta «che cosa è successo a un'istanza tra una scansione e l'altra». Comprende un identificatore, il riferimento all'PIIInstance, il change_type (vocabolario chiuso, cfr. §5.4.5), il riferimento alla Scan che ha prodotto la variazione, il riferimento alla previous_scan per lo stesso documento — ciò che lega le rilevazioni nel tempo — e un timestamp.

Le relazioni tra le entità sono: Document (1)–(N) PIIInstance (*contiene*); PIIInstance (1)–(N) PIIChange (*ha storia*); Scan (1)–(N) PIIChange (*genera*); PIIInstance (N)–(0..1) ProcessingActivity (*corrisponde a*, con 0 = orfana). La Figura 5.4 ne dà una sintesi grafica.

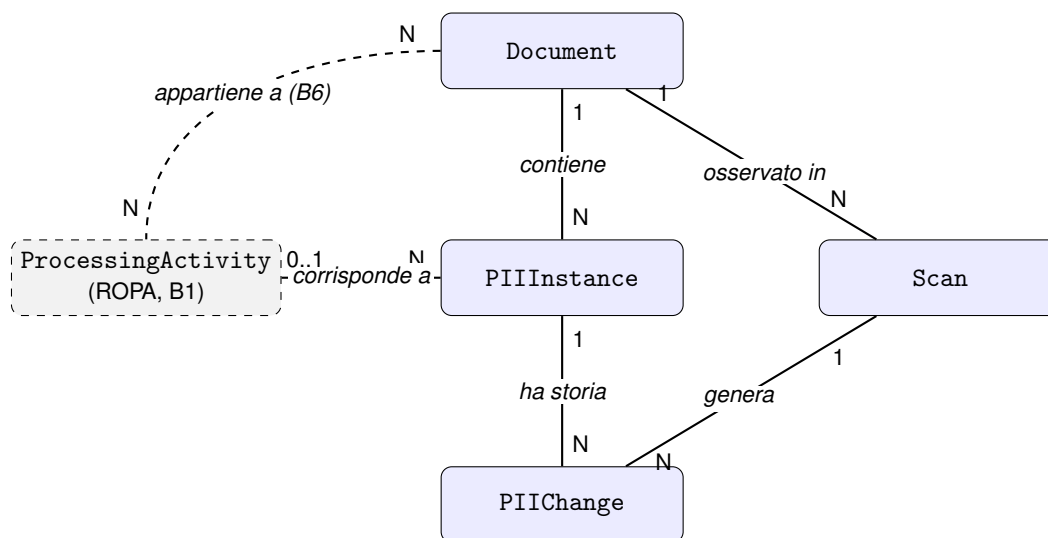


Figura 5.4: Diagramma entità-relazione del registro delle PII rilevate (blocco B5): lo stato corrente (PIIInstance) e la sua storia (PIIChange), legati alle scansioni (Scan) e ai documenti (Document). I legami con ProcessingActivity del registro ROPA (tratteggiati, blocco B1) sono due: l'associazione *documento-trattamento* (N:N, blocco B6), registrata su Document.activity_ids con la sua provenienza (association_source); e il riferimento *per singola PII* (0..1) a un trattamento giustificante, la cui assenza segnala una PII *orfana*, popolato dalla verifica di conformità (B7) a partire dall'associazione stessa. Non mostrata la tabella delle regole cartella→trattamento (registry_folder_rule, B6), che alimenta l'associazione per prefisso.

5.4.5 Risoluzione dei punti aperti di progettazione

In fase di progettazione alcune decisioni sul modello erano state lasciate deliberatamente aperte, ciascuna con un impatto concreto sul comportamento del registro. La realizzazione del blocco B5 le ha risolte come segue; le scelte sono documentate qui perché fanno parte del *modello*, non solo della sua implementazione.

5.4.5.0.1 Registrazione della «scomparsa» di una PII: esplicita. Quando una scansione non ritrova più un'istanza confermata in precedenza, la registrazione poteva essere

implicita — nessun evento, con lo stato «rimossa» dedotto confrontando l'ultima variazione dell'istanza con la data dell'ultima scansione — oppure **esplicita**. Si è adottata la modalità **esplicita**: l'istanza viene marcata come rimossa e si registra attivamente una `PIIChange` di tipo `REMOVED`. È la scelta più solida per un log autoesplicativo — l'assenza di un dato non deve richiedere conoscenza esterna per essere interpretata — e non comporta costo aggiuntivo, poiché il confronto attivo con lo stato precedente fa comunque parte del meccanismo di rilevazione dei delta dell'opzione C. L'istanza rimossa *non* viene cancellata: la riga resta, così la storia non si perde.

5.4.5.0.2 Significato dell'evento «modifica»: scomposto, nessun `MODIFIED`. Poiché il registro non salva i valori effettivi delle PII, un evento di «modifica» non può riferirsi al cambiamento del *contenuto*, mai memorizzato. Si è quindi deciso di **non** prevedere un generico evento `MODIFIED` e di scomporlo in variazioni dal significato preciso. Il solo spostamento nel documento, a parità di tipo, è un evento `MOVED`; un cambiamento di *tipo*, invece, non è modellato come riclassificazione in loco, ma emerge come `REMOVED` dell'istanza precedente più `NEW` della nuova — coerente con il modo in cui l'identità di un'istanza è approssimata (si veda il punto seguente).

5.4.5.0.3 Vocabolario chiuso dei tipi di variazione. Il campo `change_type` è formalizzato come enumerazione chiusa di quattro valori: `NEW` (istanza rilevata per la prima volta), `CONFIRMED` (istanza già nota, ritrovata invariata), `MOVED` (istanza già nota, ritrovata in posizione diversa) e `REMOVED` (istanza già nota, non più presente). Poiché il valore della PII non è memorizzato, l'**identità** di un'istanza fra due scansioni non può basarsi sul contenuto: viene approssimata da `pII_type` più **prossimità di posizione** — stesso tipo alla stessa posizione dà `CONFIRMED`, stesso tipo alla posizione più vicina dà `MOVED`. È questa regola pragmatica a dare un significato operativo a `MOVED` in assenza dei valori. L'insieme è completo per il changelog realizzato; un'eventuale estensione (ad esempio con l'arrivo dell'associazione ai trattamenti, B6) sarebbe un'evoluzione dell'enumerazione, non una sua ambiguità residua.

5.4.5.0.4 Ambito dello snapshot iniziale: prima scansione tutta `NEW`. Anche un modello delta-based ha bisogno di un punto di partenza: la prima scansione di un documento non ha una «scansione precedente». Il *bootstrap* è risolto registrando tutte le istanze della prima scansione come `NEW`, con `previous_scan` nullo. Ciò è coerente con quanto nota Kleppmann sul CDC, dove il processo parte comunque da uno snapshot iniziale che fa da riferimento per la successiva cattura dei delta [kleppmannDesigningDataIntensive2017].

5.5 Associazione e verifica di conformità

I blocchi B6 e B7 della Figura 5.1 sono il punto in cui le due metà del sistema — il *dichiarato* (il ROPA, §5.1) e il *rilevato* (il registro delle PII, §5.4) — si incontrano e producono il **verdetto di conformità**. Fino a questo punto il sistema sa, separatamente, quali categorie un trattamento *dichiara* e quali PII un documento *contiene*; è il confronto fra i due insiemi a trasformare un rilevatore di PII in uno strumento di *governance*, ed è ciò che distingue il progetto dagli strumenti di sola rilevazione (§3.8).

5.5.1 Associazione documento–trattamento (B6)

Il confronto ha senso solo rispetto a un termine di paragone: prima di chiedersi «questo documento è conforme?», bisogna sapere *a quale trattamento* il documento appartiene. È il compito del blocco B6: associare un documento a una o più `ProcessingActivity` dichiarate.

L'associazione è **esplicita** — la stabilisce il DPO, non un'inferenza automatica — e la scelta è metodologica, non di comodo. Un'associazione *per contenuto*, che attribuisse il documento al trattamento i cui dati dichiarati somigliano di più a quelli rilevati, sarebbe **circolare**: sceglierebbe per costruzione il trattamento che rende il documento «il più conforme possibile», svuotando di senso la verifica successiva. L'associazione esplicita mantiene invece il confronto **onesto**: il trattamento è deciso da un umano, e B7 misura poi, senza sconti, ciò che quel trattamento dichiara rispetto a ciò che il documento contiene. L'associazione resta comunque dietro un contratto (`ActivityAssigner`), così un proponente automatico o assistito dall'AI potrà in futuro *proporre* l'attribuzione lasciando al DPO la conferma — lo stesso schema *human-in-the-loop* già adottato per la mappatura del ROPA (§5.1.4).

La relazione è **molti-a-molti**: un documento può rientrare in più trattamenti (un modulo usato tanto nella selezione del personale quanto nelle paghe) e un trattamento raccoglie molti documenti. L'insieme dei trattamenti di un documento è registrato sul documento stesso (`Document.activity_ids`); poiché ROPA e registro delle PII vivono su basi dati separate (§5.4), gli identificatori di trattamento fanno da collegamento *logico* fra i due mondi, senza chiave esterna fisica.

Perché l'associazione esplicita sia **praticabile su cartelle reali** — dove attribuire un documento alla volta è improponibile — il DPO dispone di due modalità, entrambe *decise da lui*: l'associazione **diretta** di un singolo documento e le **regole cartella→trattamento**, con cui mappa una volta un *prefisso di percorso* a uno o più trattamenti; ogni documento che ricade sotto quel prefisso — i file nuovi inclusi, alla scansione successiva — *eredita* l'associazione. Una regola non è un'inferenza per contenuto (che si è appena esclusa perché circolare): resta una scelta esplicita del DPO, solo espressa per *collocazione* anziché documento per documento. La **provenienza** di ogni associazione è tracciata (`association_source`, con valori `MANUAL` e `RULE`) e il manuale **prevale**: riapplicare le regole non sovrascrive mai un'asso-

ciazione fatta a mano. Le regole si applicano come passo separato *dopo* la scansione, così una scansione schedulata associa da sé i documenti nuovi — un passo verso l'autonomia «dai una cartella e fa da solo».

5.5.2 Il confronto dichiarato–rilevato e il suo vocabolario (B7)

Fissata l'associazione, la verifica di conformità è un'operazione **insiemistica**. Da un lato l'insieme dei *pii_type dichiarati*: l'unione delle categorie dichiarate da *tutti* i trattamenti a cui il documento è associato (si contano le mappature confermate dal DPO, §5.1.4). Dall'altro l'insieme dei *pii_type rilevati*: i tipi delle PII effettivamente presenti nel documento secondo il registro. Confrontando i due insiemi, ogni tipo ricade in una voce di un **vocabolario chiuso**, che è il linguaggio con cui il sistema riporta il verdetto al DPO:

Coperta un tipo *rilevato* che è anche *dichiarato* da almeno uno dei trattamenti associati. È il caso conforme: il dato presente è previsto dal registro.

Orfana un tipo *rilevato* che **nessuno** dei trattamenti associati dichiara. È il segnale di conformità cardine: un dato personale presente nel documento che il ROPA non prevede — un trattamento di fatto non dichiarato (art. 30 GDPR, art. 12 nLPD). Si dice «orfana» perché priva di un trattamento-genitore che la giustifichi.

Mancante un tipo *dichiarato* da un trattamento associato ma **non** riscontrato nel documento. Non è di per sé una violazione: può indicare una sovra-dichiarazione, oppure semplicemente che quel documento non contiene quel dato.

Non risolvibile una categoria *dichiarata* nel ROPA che non si risolve in alcun *pii_type* (insieme vuoto, §5.1.3): è dichiarata ma non rilevabile dal motore. Resta esplicita e va segnalata al DPO, invece di essere scambiata per conforme.

Sotto la cardinalità molti-a-molti la definizione di orfana è deliberatamente **permissiva**: un tipo è orfano solo se *nessuno* dei trattamenti associati lo dichiara (unione, non intersezione). Un dato coperto anche da un solo trattamento pertinente è giustificato. Specularmente, a ogni singola PII rilevata la verifica assegna il primo trattamento associato che ne dichiara il tipo come «trattamento giustificante» (*processing_activity_id*), oppure nessuno se la PII è orfana: è il momento in cui il riferimento opzionale (0..1) lasciato vuoto dal registro (§5.4.4) viene finalmente popolato.

Due principi ereditati dal resto del sistema governano la lettura del verdetto. Il primo è **recall-first** (§5.3.2): il sistema preferisce segnalare un'orfana di troppo che mancarne una, perché un falso positivo costa al DPO una verifica, mentre una PII non dichiarata e non vista è un rischio invisibile. Il secondo è che il verdetto è un **segnale, non una sentenza**: un'orfana afferma «non risulta dichiarata», non «è certamente una violazione». Può infatti nascere da una violazione reale oppure da un buco di mappatura a monte — una categoria

del ROPA non ancora risolta sul tipo corrispondente. È il DPO, *human-in-the-loop*, a chiudere il giudizio; il sistema gli mette davanti l'evidenza strutturata.

5.5.3 Verifica della conservazione

Alla verifica delle categorie si affianca un controllo di **conservazione** (*retention*): un dato può essere legittimamente dichiarato eppure trattenuto oltre il periodo consentito. Il ROPA porta, a livello di macro-categoria, una durata massima *calcolabile a macchina* (`retention_months`, §5.1.3); confrontandola con l'età del documento si segnala una categoria conservata oltre il limite quando un suo dato è ancora presente. La misura dell'età è, allo stato attuale, **approssimata**: si assume come data di riferimento del documento la sua ultima modifica sul file system (il *mtime*). È un'approssimazione consapevole — una data-documento più affidabile, ricavata dai metadati in fase di estrazione, è un affinamento previsto — ma sufficiente a produrre il segnale «conservazione superata» accanto a quelli sulle categorie.

Capitolo 6

Implementazione

6.1 Ingestione del ROPA (blocco B1)

Il blocco B1 realizza il percorso che porta un registro dei trattamenti, così come lo compila un'organizzazione, alla rappresentazione interna descritta nel modello di dominio (§5.1). La realizzazione segue il principio *sottrattivo* già motivato in architettura — del documento sorgente si conserva soltanto ciò che alimenta la verifica di conformità — e lo traduce in una catena di tre passi disaccoppiati (lettura, interpretazione, persistenza), seguiti da una passata separata di mappatura assistita. Ogni passo è un modulo a sé, così che un cambiamento di formato del file o di strategia di mappatura non tocchi gli altri.

6.1.1 Il formato sorgente: il template CNIL reale

La fonte non è un formato inventato per comodità, ma il **modello di registro ufficiale della CNIL**, l'autorità francese per la protezione dei dati: un foglio di calcolo (ODS/XLSX) con una scheda operativa (*fiche*) per ciascun trattamento, oltre a schede di servizio (tutorial, elenco dei trattamenti, un modello vuoto da replicare, liste di supporto). Ancorare l'ingestione a un artefatto reale, anziché a un file cucito sul parser, rende la funzionalità dimostrabile su un input che un'organizzazione userebbe davvero. Nella *fiche*, l'ingestione modella **una sola sezione** — «Categories of personal data», con le sue colonne *categoria*, *descrizione* e *periodo di conservazione* — e scarta deliberatamente tutte le altre (attori, finalità secondarie, categorie di interessati, destinatari, trasferimenti, misure di sicurezza), coerentemente con la scelta sottrattiva. È questa sezione, e solo questa, a fornire il dichiarato che B7 confronterà con il rilevato.

6.1.2 Lettura: un'unica forma per ODS ed Excel

Il primo passo (*sheet_reader*) legge una scheda del foglio in una griglia di stringhe, dietro un'interfaccia **indipendente dal formato** (*Adapter*): l'estensione del file decide il *back-end* — *.ods* tramite *odfpy*, *.xlsx/.xlsm* tramite *openpyxl* — ma entrambi restituiscono la stessa

struttura, così che il passo interpretativo a valle ignori del tutto la provenienza. Un dettaglio pratico del formato ODS ha richiesto attenzione: nel modello CNIL diverse celle portano un **commento** d'aiuto (il «triangolino rosso»), che nel documento risiede *dentro* la cella come annotazione. Letto ingenuamente, quel testo verrebbe concatenato davanti al valore reale (per esempio «Cf. Article 87...Social Security Number»); il lettore esclude perciò i paragrafi contenuti nelle annotazioni, restituendo il solo valore della cella. È una piccola scelta di robustezza, ma tocca proprio la sezione che si vuole conservare, e come tale è coperta da test sul file CNIL reale.

6.1.3 Interpretazione: guidata dalle etichette, non dalle posizioni

Il secondo passo (*normalizer*) trasforma la griglia in un'attività di trattamento. La regola cardine è che le informazioni si cercano per **etichetta**, non per indice di riga: il nome del trattamento è il valore accanto alla cella «Name of the processing operation», la finalità quello accanto a «Main purpose», e le categorie sono le righe che seguono l'intestazione «Categories of personal data» fino alla prima riga vuota. Così la mappatura **sopravvive** a spostamenti di righe e alla presenza delle numerose sezioni che non si modellano: il parser vi passa sopra senza inciampare. La stessa regola fa da filtro naturale sulle schede di servizio: una scheda priva del nome di trattamento — il tutorial, le liste, il modello *vuoto* — non è un'attività e viene saltata sollevando un errore che il chiamante intercetta, sicché un intero *workbook* CNIL produce esattamente un'attività per ogni *fiche* compilata. Il periodo di conservazione, infine, viene reso **calcolabile**: la dicitura originale («5 years from the payment of the salary») è conservata per audit accanto a una durata in mesi estratta da essa (60), quando il testo esprime una durata fissa; resta *None* quando esprime un criterio, coerentemente con il campo del modello (§6.3.1).

6.1.4 Persistenza e mappatura assistita

Il terzo passo affida l'attività così ottenuta al ROPARepository, che la scrive nella base dati ROPA riusando lo stesso modello di dominio come schema (*Active Record*, §6.3). La **risoluzione delle categorie sui pii_type** del catalogo condiviso — l'aggancio verso il rilevamento — è invece una passata **separata e successiva**, non parte del parsing deterministico: per ogni categoria ancora non risolta, una strategia dietro il contratto CategoryMapper propone i pii_type corrispondenti al suo testo libero. La strategia è intercambiabile — un **dizionario** deterministico (parole chiave), un **modello generativo** (§6.5) o una combinazione dei due — e la passata è **idempotente**: ciò che il mapper non sa risolvere resta invariato, così una riesecuzione mappa solo ciò che è nuovo. Il risultato è comunque una *proposta*: lo stato del mapping resta PROPOSED finché il DPO non lo conferma (*human-in-the-loop*, §5.1.4), a ribadire che l'AI arricchisce ma non decide.

L'interazione del DPO avviene tramite un'interfaccia web dedicata (blocco B8, la review montata sotto /ropa), da cui può confermare le mappature, **importare** un nuovo registro caricandone il file dal browser — il file viaggia verso il servizio come *byte* via *multipart*, coerente con il deployment containerizzato in cui l'applicazione non accede al file system del client — e **rimuovere** un trattamento. L'import riusa senza modifiche la catena di ingestione appena descritta: la stessa lettura, la stessa interpretazione, la stessa mappatura.

6.2 Tecnologia di persistenza

Il modello concettuale della persistenza — il *changelog ibrido delta-based* per il registro delle PII (§5.4) e la gerarchia del registro ROPA (§5.1) — è *technology-agnostic*: restava da fissare la tecnologia concreta con cui realizzarlo. I candidati erano due:

- una base dati **relazionale** (SQLite), che offre *join* e interrogazioni relazionali «gratuite» ed è coerente col deployment locale in container senza servizi esterni;
- una persistenza **su file semi-strutturati** (JSON/JSONL), più leggera ma priva di garanzie relazionali native.

La scelta è ricaduta su **SQLite**, per ragioni che discendono dalla natura del dato e dai vincoli di progetto:

- **Il dato è intrinsecamente relazionale.** Entrambi i registri sono grafi di entità legate da chiavi esterne (documento–scansione–istanza–variazione; attività–macro-categoria–categoria) e le interrogazioni che servono ai blocchi a valle — stato corrente di un documento, storia di un'istanza, PII orfane, confronto dichiarato-vs-rilevato — sono *join* e operazioni insiemistiche che un motore relazionale risolve nativamente. Su file JSON andrebbero reimplementate a mano, riscrivendo ciò che un DBMS offre già.
- **SQLite è *embedded*, senza server.** È un singolo file, nessun processo o servizio esterno da gestire: coerente col deployment containerizzato e, soprattutto, con la **minimizzazione del dato** — i dati non lasciano l'infrastruttura on-premise e nessun servizio di terzi è coinvolto.
- **Una sola definizione (*Active Record*).** Con SQLAlchemy l'entità di dominio è la tabella (§6.3): niente coppia dominio/riga né traduzione a mano, coerente col principio DRY.
- **La porta resta aperta.** SQLAlchemy poggia su SQLAlchemy: migrare a un RDBMS in rete (ad esempio PostgreSQL), qualora si superi il singolo nodo, cambia l'URL di connessione — letto da variabile d'ambiente (§6.6) — e non il modello dei dati. SQLite è dunque la scelta adeguata *ora*, non un vicolo cieco.

La stessa tecnologia serve entrambi i registri, su due basi dati separate (variabili d'ambiente ROPA_DB_URL e PII_DB_URL); *che cosa* vi si conserva, campo per campo, è dettagliato nel §6.3.

6.3 Dati persistiti nei database

Il sistema mantiene due basi dati distinte e separate (§6.6): il **registro ROPA** (blocco B1) e il **registro delle PII rilevate** (blocco B5). Ogni entità è definita una sola volta come tabella SQLModel — il modello di dominio è lo schema di persistenza (*Active Record*) — così da non duplicare la struttura. I modelli concettuali e le relazioni sono discussi in §5.1.3 e §5.4.4; questa sezione documenta, **campo per campo, ciò che viene effettivamente scritto su disco e perché**. La realizzazione corrente è su SQLite (§6.2), ma il modello dei dati qui descritto è indipendente dalla tecnologia sottostante.

Un principio attraversa entrambe le basi ed è la chiave di lettura di ogni campo che segue: la **minimizzazione del dato** (§2.3.11). In particolare il registro delle PII conserva soltanto *riferimenti* — tipo, posizione, provenienza, documento — e **mai il valore** della PII, come si vedrà nell'assenza deliberata di una colonna che lo contenga.

6.3.1 Registro ROPA (blocco B1)

Tre tabelle rispecchiano la gerarchia a tre livelli del registro CNIL (§5.1.3).

6.3.1.0.1 processing_activity — l'attività di trattamento (livello 1).

id identificatore stabile dell'attività (chiave primaria), assegnato dal chiamante; ancora dell'intera sotto-struttura.

name nome del trattamento, per leggibilità e report al DPO.

purpose finalità dichiarata del trattamento, elemento cardine dell'art. 30 GDPR.

6.3.1.0.2 macro_category — il blocco di categorie con la sua conservazione (livello 2).

id chiave surrogata (auto-generata).

activity_id chiave esterna all'attività: colloca la macro-categoria nella gerarchia.

raw_text dicitura originale della macro-categoria, conservata *tale e quale* per auditabilità (nulla del documento sorgente va perduto).

retention_text dicitura originale del periodo di conservazione, conservata per audit.

`retention_months` la stessa conservazione resa **calcolabile dalla macchina** (durata in mesi), affinché il controllo di retention (B7) sia un confronto e non una lettura di prosa; vale None quando il registro esprime un criterio anziché una durata fissa.

6.3.1.0.3 `declared_category` — la categoria dichiarata, ponte verso il rilevamento (livello 3).

`id` chiave surrogata.

`macro_category_id` chiave esterna alla macro-categoria di appartenenza.

`raw_text` testo libero originale della categoria (es. «nome»), conservato per audit.

`pii_types` l'insieme di `pii_type` del catalogo condiviso a cui la categoria è ricondotta: è l'**aggancio** che rende il confronto fra ciò che il ROPA *dichiara* e ciò che il motore *rileva* (B7) una semplice operazione insiemistica.

`mapping_state` stato del mapping (PROPOSED dall'AI / CONFIRMED dal DPO): materializza il *human-in-the-loop*: l'AI propone la corrispondenza, ma non la fissa mai da sola.

6.3.2 Registro delle PII rilevate (blocco B5)

Quattro tabelle realizzano il *changelog* delta-based (§5.4.4): stato corrente più storia. Una quinta tabella, `registry_folder_rule`, risiede nella stessa base dati ma appartiene al blocco di associazione (B6, §5.5.1) ed è descritta in coda a questa sezione. È nel *changelog* che la minimizzazione si vede nel dettaglio.

6.3.2.0.1 `registry_document` — il documento analizzato.

`document_id` identificatore stabile del documento (chiave primaria), coerente con il `document_id` prodotto da B3; nella scansione ricorsiva di una cartella è il **percorso relativo** (PO-SIX) alla radice, ciò che consente alle regole cartella→trattamento di combaciare per prefisso (§5.5.1).

`path` percorso originale del file, come riferimento; opzionale.

`first_seen` istante di prima registrazione del documento.

`activity_ids` gli identificatori dei trattamenti dichiarati a cui il documento è associato (B6): è una **lista** (cardinalità N:N, §5.5.1) e i suoi valori riferiscono `ProcessingActivity` nell'altra base dati, come collegamento *logico* senza chiave esterna fisica.

`association_source` la **provenienza** dell'associazione — MANUAL (impostata dal DPO) oppure RULE (derivata da una regola cartella→trattamento) — oppure None se il documento non è ancora associato. È il campo che fa **vincere il manuale sulle regole**: riapplicare le regole salta i documenti a provenienza MANUAL.

`source_modified_at` la data di ultima modifica del file sorgente (il suo `mtime`) al momento dell'ingestione, usata come data di riferimento del documento per il controllo *approssimato* di conservazione (B7, §5.5.3); è un *riferimento*, non un valore di PII. Vale `None` se ignota.

Il vocabolario di `association_source` è un'enumerazione chiusa (`AssociationSource: MANUAL/RULE`), come per gli altri campi a dominio fissato del modello.

6.3.2.0.2 `registry_scan` — una scansione, cioè un'osservazione nel tempo.

`id` chiave surrogata.

`document_id` chiave esterna al documento osservato.

`created_at` istante della scansione: è l'**asse temporale** su cui poggia l'intero changelog.

6.3.2.0.3 `registry_pii_instance` — la singola PII, stato corrente e mutabile. Nessuna colonna di questa tabella contiene il *valore* della PII: è il punto in cui la minimizzazione diventa concreta.

`id` chiave surrogata.

`document_id` chiave esterna al documento che la contiene.

`pii_type` la categoria del catalogo condiviso (es. `iban`) — *non* il valore.

`start`, `end` la **posizione** come intervallo di caratteri nel testo normalizzato: un riferimento a *dove* si trova la PII, non al suo contenuto.

`confidence` confidenza aggregata dal merge, per consentire al DPO ordinamento e soglie.

`confirmation_level` esito del merge (`single_source` / `double_confirmed` / `conflicting` / `ai_discovered`): traccia *quanto* è certa la rilevazione.

`sources` gli identificatori dei detector che l'hanno trovata: traccia *chi* l'ha rilevata (tracciabilità, §2.7.3).

`processing_activity_id` l'attività dichiarata a cui la PII corrisponde, oppure `None` se la PII è **orfana** (non riconducibile ad alcun trattamento); popolata dal blocco di associazione (B6), senza chiave esterna fisica finché i due database restano distinti.

`last_scan_id` chiave esterna all'ultima scansione che l'ha osservata.

`removed` flag di stato: `True` se l'istanza non è più presente, marcata dalla logica di delta; la riga resta, così la storia non si perde.

L'**assenza** di un campo `value/text` è una scelta di progetto, non una dimenticanza: garantisce che un registro di conformità non si trasformi a sua volta in un archivio di dati personali.

6.3.2.0.4 registry_pii_change — il log append-only delle variazioni.

id chiave surrogata.

pii_instance_id chiave esterna all'istanza cui la variazione si riferisce.

change_type il tipo di variazione, vocabolario chiuso: NEW / CONFIRMED / MOVED / REMOVED (§5.4.5).

scan_id chiave esterna alla scansione che ha prodotto la variazione.

previous_scan_id chiave esterna alla scansione *precedente* dello stesso documento — ciò che lega le rilevazioni nel tempo; vale None alla prima scansione (*bootstrap*).

timestamp istante in cui la variazione è stata registrata.

6.3.2.0.5 registry_folder_rule — la regola cartella→trattamento (blocco B6). Fisicamente nella stessa base dati del registro delle PII, ma logicamente parte dell'associazione (§5.5.1): mappa un prefisso di percorso ai trattamenti che i documenti sotto di esso ereditano.

prefix il prefisso di percorso (POSIX, normalizzato senza slash finale; "" è la radice e combacia con ogni documento). Chiave primaria.

activity_ids i trattamenti con cui associare i documenti che cadono sotto prefix; riferiscono ProcessingActivity nell'altra base dati (nessuna chiave esterna fisica), come Document.activity_ids.

created_at istante di creazione della regola.

In sintesi: il registro ROPA conserva il dichiarato (con il testo originale accanto alla forma strutturata, per audit), mentre il registro delle PII conserva soltanto *riferimenti* a ciò che è stato rilevato e la loro storia — mai i valori. Le due basi restano separate; i campi Document.activity_ids (associazione N:N, B6) e PIIInstance.processing_activity_id (trattamento giustificante per singola PII, popolato da B7) sono i punti in cui i due mondi si collegano *logicamente*, sull'identificatore-stringa del trattamento.

6.4 Riutilizzo di un motore di detection esistente: valutazione di Microsoft Presidio

Come emerso dallo stato dell'arte (§3.8), Microsoft Presidio non risolve il problema di governance affrontato dal progetto; il suo *motore di rilevamento*, però, copre gran parte del livello regex e NER che il sistema deve comunque realizzare. Si valuta quindi se **costruire tale livello sopra Presidio** anziché scriverlo da zero. La decisione è affidata a

una **valutazione empirica**: questa sezione ne fissa i termini, mentre l'esito — l'adozione di Presidio, con GLiNER per il livello NER — è riportato nel §7.1.

Due modelli di integrazione sono possibili, con implicazioni opposte sull'architettura:

- usare l'*AnalyzerEngine* di Presidio come **orchestratore** dell'intera detection: massimizza il riuso, ma sostituirebbe la pipeline ibrida e la logica di merge proprietarie descritte nel Capitolo 5, che costituiscono il contributo del progetto;
- incapsulare Presidio **dietro il contratto** `PIIDetector` come un singolo detector che ne mappa i risultati in `PIICandidate`: la pipeline, il `MergeEngine` e il detector AI selettivo restano invariati, e Presidio diventa solo il fornitore di regex, checksum e NER.

Il secondo modello è preferito, perché preserva l'architettura e rende la scelta reversibile: l'astrazione `PIIDetector` consente di sostituire il fornitore senza toccare il resto del sistema. I criteri di valutazione sono:

- **copertura** delle categorie del catalogo: Presidio fornisce riconoscitori pre-costruiti per gran parte di esse (email, IBAN, carta di credito, indirizzo IP, codice fiscale italiano), diversi con **validazione a checksum** già inclusa — proprio ciò che il progetto aveva rimandato;
- **estensibilità** sugli identificatori mancanti: il numero AVS svizzero richiederebbe comunque un riconoscitore su misura, come nell'implementazione attuale;
- **integrazione del livello NER**: Presidio supporta ufficialmente modelli esterni, incluso GLiNER, coerente con la scelta zero-shot (§3.1.3);
- **vincoli di progetto**: esecuzione interamente on-premise e configurabilità da file, coerente con l'estensibilità config-driven richiesta.

6.4.0.0.1 Profilo hardware. Un ulteriore elemento a favore riguarda i requisiti di calcolo. Il motore di Presidio e il livello di rilevamento basato su spaCy operano interamente su **CPU**, senza richiedere una GPU dedicata; l'accelerazione a GPU è soltanto opzionale ed è riservata ai modelli NER più onerosi (transformers, GLiNER), verso i quali il sistema ripiega comunque in automatico sulla CPU qualora la GPU non sia presente. Questo profilo è coerente con il vincolo di esecuzione on-premise su infrastruttura di PMI, dove hardware accelerato è raramente disponibile, e costituisce quindi un rafforzamento del posizionamento del progetto più che una limitazione.

Restano in ogni caso a carico del progetto, indipendentemente dall'esito: il riconoscitore per il numero AVS, il `MergeEngine` con i livelli di conferma, il detector AI selettivo e l'intero blocco di ingestione ROPA e verifica di conformità.

Il giudizio è affidato a una **misurazione diretta**: Presidio viene incapsulato come detector e valutato sullo stesso corpus annotato e con la stessa procedura di scoring (precision, recall,

F1) impiegati per la baseline regex, così da confrontare i due approcci sullo stesso metro; l'esito del confronto è nel §7.1.

6.4.1 Estensibilità config-driven: aggiungere categorie e pattern

Una proprietà di progetto del livello di rilevamento è che il suo vocabolario non è cablato nel codice ma **dichiarato in configurazione**. Il catalogo delle categorie di PII (`categories.yaml`) e i pattern a espressioni regolari personalizzati (`custom_patterns.yaml`) sono file validati al caricamento; aggiungere una nuova categoria o un nuovo identificatore strutturato — un codice aziendale, un riferimento interno, un numero di settore — è quindi una **modifica di configurazione, non di codice**. Le regole così dichiarate vengono eseguite a runtime dal riconoscitore generico (`RegexDetector`) e affiancate ai riconoscitori pre-costruiti di Presidio tramite un `CompositeDetector`, così da presentarsi al merge come un'unica sorgente di pattern.

La scelta separa *ciò che* si rileva — competenza del DPO o del manutentore, in configurazione — da *come* lo si rileva, cioè il motore, che resta stabile. Il file dei pattern personalizzati è riservato agli identificatori che Presidio non copre nativamente (il numero AVS svizzero ne è il primo esempio), per non duplicare ciò che il motore già fornisce. La validazione al caricamento — il pattern deve compilare, ogni `pII_type` deve esistere nel catalogo — fa fallire subito una configurazione errata, invece di degradare il rilevamento in silenzio. Resta fuori, come sviluppo dichiarato, la validazione a *checksum* degli identificatori: un pattern è riconosciuto per forma, non verificato nel contenuto.

6.5 Scelta dello Small Language Model per gli impieghi selettivi dell'AI

Il progetto fa dell'AI generativa un uso **selettivo**: l'uso di LLM è esclusa dal rilevamento primario ricorrente (blocco B4) per ragioni di costo energetico [49] e riservata a compiti mirati e non ricorrenti. Sono tre i compiti candidati: (1) la **normalizzazione del ROPA** (§5.1.4) — ricondurre descrizioni sbrigative («dati anagrafici») a un vocabolario chiuso di categorie, cioè estrazione di informazione strutturata a output vincolato; (2) l'**analisi profonda di documenti campione** (tipicamente 1 su 100/1000, dove la latenza non è vincolante); (3) l'**analisi della struttura del file system** per individuare collocazioni anomale (ad esempio un curriculum in una cartella `turni/`). I vincoli discendono dal progetto: il modello deve essere **piccolo, eseguibile su CPU, containerizzabile e scalabile** su hardware commodity. Si adotta la definizione operativa di *Small Language Model* (SLM) del survey di riferimento — modelli *decoder-only* da 100M a 5B parametri [50]. Come per Presidio (§6.4), la scelta è **empirica**: seguono i criteri, le misurazioni raccolte in letteratura e la conclusione su quale modello sia il migliore.

6.5.1 Criteri di selezione

I criteri, in ordine di priorità per il caso d'uso, e le fonti metodologiche che li sostengono sono riassunti nella Tabella 6.1.

Tabella 6.1: Criteri di selezione dello SLM e relative fonti.

Criterio	Motivazione nel progetto	Fonte
Eseguibilità su CPU (throughput, memoria)	Ambiente target senza GPU	[51]
Quantizzabilità (GGUF / llama.cpp)	Stare in RAM commodity e in Docker	[52, 53]
<i>Instruction following</i> / output strutturato	Task 1 e 3 (aderenza a schema/istruzioni)	[54, 55]
Licenza permissiva + on-premise	Sovranità del dato (dati sensibili, nLPD)	[56]
Contesto lungo	Task 2 e 3 (documenti/alberi ampi)	[57, 58]

6.5.2 Fattibilità dell'inferenza su CPU

La fattibilità dell'inferenza su sola CPU non è un'assunzione ma un risultato empirico. La **quantizzazione a 4 bit** riduce l'occupazione di memoria di circa $4\times$ rispetto a `float16` con perdita di accuratezza mediamente inferiore all'1% [52]; nel formato GGUF (K-quant Q4_K_M), uno studio su hardware consumer riporta il passaggio da 2.6 a 47.9 token/s (circa $18\times$) con la RAM che scende da 14.39 GB a $\sim 1.15\text{--}1.51$ GB (oltre il 90% in meno) [59], e un secondo studio conferma su CPU uno *speedup* di $1.5\text{--}2.5\times$ da F16 a Q4 [60]. Il motore di inferenza di riferimento è `llama.cpp` (grafo compilato in C++, formato GGUF a file singolo), che il progetto serve concretamente tramite **Ollama** — un server che incapsula `llama.cpp`, espone i modelli GGUF su porta ed è configurato da variabile d'ambiente (§6.6.2); le alternative orientate alla GPU non sono adatte, poiché il backend CPU di vLLM è stato misurato a 3.07 token/s su un modello da 0.5B e va in *out-of-memory* oltre 1.5B [51]. Il costo va dichiarato: su Llama-3.2-3B l'MMLU passa da 64.2 a 61.8 con GGUF Q4_K_M [61].

Lo studio più direttamente trasferibile al progetto misura sette modelli *instruction-tuned* quantizzati a 4 bit su tre ambienti CPU-only, in un runtime containerizzato in Docker [52]. I dati di throughput e di memoria sono riportati nelle Tabelle 6.2 e 6.3 (E5-2695 v2 = Xeon 12-core del 2013 senza AVX2/AVX-512; 8480+ = Xeon Sapphire Rapids 56-core; i7-13700H = laptop 14-core).

Tabella 6.2: Throughput (token/s), 4-bit GGUF su CPU [52].

Modello	Param.	E5-2695 v2	Xeon 8480+	Core i7-13700H
Gemma-3-1B-it	1B	30	100	70
Llama-3.2-1B-Instruct	1.2B	25	120	80
Phi-4-mini-instruct	3.8B	15	80	40
Qwen2.5-7B-Instruct	7.6B	8	45	15
DeepSeek-R1-Distill-Llama-8B	8B	7	40	10
Gemma-3-12B-it	12B	5	30	9
Mistral-Small-3.1-24B	24B	2	15	4

Tabella 6.3: Occupazione di memoria: FP16 vs 4-bit (Q4) e RAM minima [52].

Modello	Param.	FP16 (GB)	Q4 (GB)	RAM min.
Llama-3.2-1B-Instruct	1.2B	~2.4	0.5	4 GB+
Gemma-3-1B-it	1B	~2.0	0.6	4 GB+
Phi-4-mini-instruct	3.8B	~7.6	1.9	8 GB+
Qwen2.5-7B-Instruct	7.6B	~15	3.7	8 GB+
Gemma-3-12B-it	12B	~24	6.7	12 GB+

I modelli 1–4B offrono interazione quasi in tempo reale su CPU; i 6–8B sono il «punto dolce» qualità/latenza su hardware datato. Per la scalabilità, oltre 8–16 thread i ritorni sono marginali: conviene eseguire **più istanze in parallelo** su gruppi di core disgiunti (un thread per core fisico, con *pinning* NUMA) piuttosto che una singola istanza su tutti i core [52]. Le versioni *instruction-tuned* sono essenziali. Il caso di studio dell'autore fa co-risiedere due modelli piccoli in Docker su una VM da 16 vCPU / 32 GB con latenza mediana inferiore a 2 s su Xeon moderno — l'analogo diretto del deployment previsto qui [52].

6.5.3 Modelli candidati

6.5.3.0.1 Qwen2.5 (Alibaba). Taglie 0.5B–72B con versioni quantizzate ufficiali; pre-addestramento su 18T token e supporto multilingue (29+ lingue) [62]. La generazione precedente offre numeri puntuali (Qwen2-1.5B: MMLU 56.5, GSM8K 58.5, HumanEval 31.1) [63]. La granularità di taglie (0.5B/1.5B/3B) lo rende tra i più flessibili per CPU.

6.5.3.0.2 Llama 3.2 1B/3B (Meta). Costruiti con *pruning* e distillazione per l'*edge*, contesto fino a 128K token; sul 3B, valori auto-riportati: IFEval 77.4, MMLU 63.4, *tool use* BFCL V2 67.0 (contro 25.7 del 1B) [58]. È il più efficiente in energia/latenza della sua fascia (§6.5.6).

6.5.3.0.3 Phi-3-mini / Phi-4-mini (Microsoft). Phi-3-mini (3.8B) raggiunge MMLU 69% e MT-bench 8.38 pur girando su telefono [64]; Phi-4-mini (3.8B) aggiunge vocabolario a 200K token, *grouped-query attention* e **instruction following e function calling nettamente migliorati**, con matematica/codice pari a modelli di taglia doppia [abdin2024phi4, 65]. È il candidato più forte nella fascia ~4B per output vincolato.

6.5.3.0.4 Gemma 3 (Google). Famiglia 1B–27B con distillazione, contesto $\geq 128K$ e KV-cache alleggerita; Gemma3-4B-IT è dichiarato competitivo con Gemma2-27B-IT dopo il nuovo post-training [57].

6.5.3.0.5 SmoLLM2 (Hugging Face). Famiglia 135M/360M/1.7B completamente aperta; SmoLLM2-1.7B supera Qwen2.5-1.5B di ~6 punti su MMLU-Pro [66]. Opzione minimale/riproducibile, meno adatta a ragionamento complesso.

6.5.3.0.6 IBM Granite 3.x. Famiglia dense 2B/8B e MoE 1B/3B sotto licenza Apache 2.0, progettata per l'impresa e l'on-premise (RAG, classificazione, *entity extraction*, *tool use*); i modelli MoE sono indicati per CPU/edge, e la pipeline documentale usa Docling — lo stesso toolkit del blocco B3 [56]. Rilevante per sovranità del dato e coerenza di ecosistema.

6.5.4 Analisi per task

6.5.4.0.1 Task 1 — ROPA verso categorie. È estrazione a output vincolato. Il benchmark *LLMStructBench* valuta 22 modelli aperti (da 0.6B a 70B) su 995 campioni verificati manualmente e conclude che **la strategia di *prompting* conta più della taglia**, e che fornire lo **schema JSON nel prompt** è l'opzione più sicura per output parsabili, specialmente sui modelli piccoli [54]. Ciò richiama la distinzione tra output *corretto* e *utilizzabile*: una risposta che viola lo schema è inservibile quanto una sbagliata [67]. Ne discende la raccomandazione: non il modello più grande, ma *constrained decoding*/schema nel prompt più un modello con forte *instruction following* — nella fascia CPU i migliori sono Phi-4-mini e Llama-3.2-3B; per formati verificabili è pertinente IFEval-FC [55].

6.5.4.0.2 Task 2 — Analisi di documenti campione. Cadendo il vincolo di throughput, dominano ragionamento e contesto lungo; lavorando sul contenuto fornito (setting *RAG*) la conoscenza parametrica pesa meno e anche modelli piccoli rendono bene [52]. È quindi lecito salire di taglia: un 7B (Qwen2.5-7B) o Phi-4-mini restano eseguibili su CPU (~45 e ~80 token/s su Xeon moderno).

6.5.4.0.3 Task 3 — Struttura del file system. **Lacuna dichiarata:** non è stato individuato un benchmark che misuri il rilevamento di anomalie su gerarchie di file system. In assenza di metrica diretta, l'ancoraggio è alle capacità più prossime — *instruction following* (Llama-3.2-3B, Phi-4-mini) e contesto lungo (modelli a 128K) — ma la validazione su un dataset costruito ad hoc resta necessaria.

6.5.5 Confronto e conclusione

Nessun singolo modello *domina* su tutti e tre i task; tuttavia, per contenere la complessità del deployment — una sola immagine, un solo peso da distribuire e mantenere, nessuna gestione di co-residenza di modelli diversi — il progetto adotta **un unico modello per l'intera pipeline**. La Tabella 6.4 mostra i candidati migliori per ciascun task; il criterio di scelta è quindi l'**intersezione**: quale modello sia adeguato a tutti e tre insieme.

Tabella 6.4: Raccomandazione per task (fascia CPU).

Task	Requisito dominante	Candidati
1 — ROPA → categorie	Output strutturato + instruction following	Phi-4-mini 3.8B , Llama-3.2-3B, Qwen2.5-3B
2 — Analisi documenti	Ragionamento + contesto lungo (latenza libera)	Qwen2.5-7B , Phi-4-mini 3.8B
3 — Struttura FS	Instruction following + contesto lungo	Llama-3.2-3B , Phi-4-mini, Gemma-3-4B

Il modello scelto è *Phi-4-mini* (3.8B). È l'unico candidato presente in tutte e tre le righe della Tabella 6.4: offre il miglior equilibrio *instruction-following/function-calling* nella fascia ~4B [65], ciò che il Task 1 e il Task 3 richiedono come requisito dominante, e resta al contempo adeguato al Task 2, dove la caduta del vincolo di throughput rende sufficiente un modello da ~4B lavorando in *RAG* sul contenuto fornito [52]. Sul piano operativo è la scelta più leggera compatibile con tutti i compiti: **1.9 GB in Q4** e ~80 token/s su Xeon moderno, un solo peso GGUF servito da un solo `llama.cpp` in Docker [52]. Rinunciare a un secondo modello dedicato (ad esempio *Qwen2.5-7B* per il ragionamento del Task 2) è un compromesso consapevole: si accetta un margine di qualità potenzialmente inferiore sul Task 2 in cambio di un deployment radicalmente più semplice e portabile, coerente con il vincolo di containerizzazione del progetto. Come **alternativa orientata a sovranità e governance**, a parità di modello unico, resta *IBM Granite 3.x* (dense 2B/8B, Apache 2.0), coerente con l'ecosistema Docling già in uso [56].

La scelta resta soggetta a **validazione empirica** sul dominio, come per Presidio: un *gold set* etichettato (per il Task 1, frasi di «categorie di dati» con l'insieme atteso di `pii_type`) su cui misurare *precision/recall/F1* a prompt fissato, riusando l'impianto di scoring del progetto. Il componente è isolato dietro il contratto *CategoryMapper*, per cui cambiare modello — o passare a un dizionario deterministico — non tocca il resto della pipeline.

6.5.6 Energia, efficienza e limiti

Sul versante *edge*, uno studio sull'impronta energetica riporta che su Raspberry Pi 5 Llama 3.2 è il più efficiente in potenza (fino a 2.05 Wh) e a più bassa latenza (4.06 s su MMLU), mentre Phi-3 Mini raggiunge l'accuratezza più alta (MMLU 62.3%) ma con la latenza peggiore (34.36 s) [68]. Esiste dunque un compromesso esplicito accuratezza–energia–latenza: poiché i tre task sono a **bassa frequenza** (non ricorrenti sull'intero corpus), è accettabile un modello più accurato e più lento — l'inverso del ragionamento applicato a B4, dove l'inferenza ricorrente esclude l'AI generativa.

limiti da riportare nel capitolo dedicato: (i) i numeri dei *technical report* sono **auto-riportati** e vanno trattati come dichiarazioni di parte, con rischio di *benchmark contamination* [abdin2024phi4]; (ii) la quantizzazione a 4 bit introduce un decadimento reale (es. MMLU 64.2→61.8), per cui sul Task 1 va valutato Q8 come compromesso [61]; (iii) per il Task 3 nessun benchmark è diretto: l'ancoraggio a *instruction following* e contesto lungo è un *proxy*; (iv) per l'output strutturato la variabile dominante è la strategia di prompt, non solo il modello [54]; (v) i throughput dipendono dalla microarchitettura (AVX-512/VNNI/AMX) e vanno riprodotti sull'hardware target [51].

6.6 Deployment containerizzato e portabilità

La containerizzazione non è un dettaglio di confezionamento, ma un **requisito di progetto** — l'installazione deve avvenire tramite un meccanismo standardizzato e riproducibile — e un corollario diretto dell'architettura: i componenti sono concepiti come **servizi raggiungibili su porta e configurati da variabili d'ambiente**, con l'applicazione nel ruolo di interfaccia leggera. È inoltre funzionale alla **minimizzazione del dato**: tutti i servizi risiedono nella rete privata dell'orchestrazione e i dati trattati non lasciano l'infrastruttura on-premise. Questa sezione fissa l'approccio di deployment adottato e ne descrive la realizzazione.

6.6.1 Portabilità e isolamento delle dipendenze

L'obiettivo operativo è una build **riproducibile su qualsiasi piattaforma**: la stessa immagine deve girare identica in sviluppo e in produzione, indipendentemente dall'architettura dell'host. L'argomento più forte a favore emerge da un problema concreto incontrato nello sviluppo. Il livello di rilevamento NER poggia su uno *stack* di apprendimento automatico pesante e fragile (PyTorch e il modello NER GLiNER); sulla macchina di sviluppo — un interprete Python x86_64 in emulazione su hardware ARM — la versione di PyTorch disponibile risultava troppo datata per la restante toolchain (NumPy, transformers), rendendo il livello NER **non costruibile in locale**. Un container Linux, con dipendenze moderne, risolve il problema alla radice e — punto generale — **disaccoppia la build dall'architettura dell'host**: ciò che in locale è un vicolo cieco diventa, nel container, l'ambiente di riferimento in cui la stessa immagine è riproducibile ovunque. La containerizzazione non è quindi soltanto un requisito formale, ma la risposta pragmatica alla fragilità delle dipendenze di apprendimento automatico.

6.6.2 Architettura di deployment

Il sistema è orchestrato come un insieme di servizi. Il **servizio applicativo** racchiude il nucleo — ingestione ROPA, motore di detection con i livelli regex/checksum e NER, verifica di conformità, interfaccia per il DPO — insieme alle relative dipendenze, comprese quelle di apprendimento automatico più onerose (Presidio, spaCy, GLiNER), che nell'immagine Linux si installano senza attriti. Il **runtime del modello generativo** — un servizio **Ollama** (basato su `llama.cpp`) che serve lo SLM scelto (§6.5) su porta, con host e nome del modello letti da variabili d'ambiente (`OLLAMA_HOST`, `ROPA_LLM_MODEL`) — è un container separato, coerente con l'uso selettivo e non ricorrente dell'AI. La **base dati** è, allo stato attuale, un file SQLite su volume persistente (§6.2). Tutta la configurazione — URL della base dati, host e nome del modello — è letta da **variabili d'ambiente**, così la stessa immagine gira in locale e in container cambiando solo la configurazione; i pesi dei modelli e lo stato della

base dati vivono su **volumi** dedicati, per non essere riscaricati o persi tra un'esecuzione e l'altra. Coerentemente col profilo hardware del progetto (§6.4), tutti i servizi operano su **CPU**.

6.6.3 Contratti e direzione

Il disaccoppiamento è garantito dai contratti già introdotti all'interno dell'architettura: `PIIDetector` per i rilevatori, `CategoryMapper` per la normalizzazione del ROPA, un client astratto per il modello generativo. Poiché ogni funzionalità dipende solo dalla *forma* del contratto e non dalla sua collocazione, **spostare una funzionalità in un sotto-container dedicato ha un costo quasi nullo**. La direzione è quindi l'articolazione progressiva in **sotto-container per funzionalità**: il runtime generativo è già un servizio a sé; il livello NER, oggi in-process, è scorporabile in un servizio dedicato qualora se ne debba scalare il carico; la persistenza migrerà da SQLite a una base dati relazionale in rete (ad esempio PostgreSQL) quando si superi il singolo nodo. Rimane come sviluppo la pubblicazione di immagini **multi-architettura** (amd64/arm64). Il container costituisce la via canonica per eseguire il sistema completo, dipendenze di apprendimento automatico incluse, mentre il nucleo leggero resta eseguibile nativamente per l'iterazione rapida di sviluppo.

Capitolo 7

Test e validazione

7.1 Validazione del livello di rilevamento

Le scelte lasciate aperte in fase di implementazione — se costruire il livello regex/NER sopra Microsoft Presidio (§6.4) e quale motore di NER adottare — sono qui chiuse su base empirica.

7.1.0.0.1 Metodo. La valutazione riusa l'impianto di scoring del progetto: un corpus annotato di documenti sintetici in cui ogni PII è marcata con il proprio `pii_type` e la sua posizione. Un rilevamento conta come vero positivo quando condivide il `pii_type` con uno span di *ground truth* e vi si sovrappone con $Intersection\ over\ Union \geq 0.5$; ogni rilevamento e ogni span sono appaiati al più una volta (*matching greedy*). Da qui si derivano precision, recall e F1 micro-mediati. Ogni configurazione è incapsulata dietro il contratto `PIIDetector` e misurata sullo stesso corpus, così da confrontarle sullo stesso metro.

7.1.0.0.2 Risultati. Il confronto è nella Tabella 7.1. **Il livello a pattern di Presidio** (espressioni regolari e checksum) supera la baseline regex proprietaria (F1 0.85 contro 0.79) con precisione perfetta e senza falsi positivi, e copre nativamente categorie che la baseline mancava; viene quindi adottato, coerentemente con la scelta di incapsularlo dietro `PIIDetector` (§6.4). Per il **livello NER**, il modello generico spaCy si rivela inadeguato: F1 0.08, con recall quasi nullo e falsi positivi (etichetta come *location* identificatori strutturati, ad esempio un IBAN). Sostituendolo con **GLiNER**, NER *zero-shot* specifico per le PII, il livello NER sale a F1 0.44 e — dato decisivo — la detection completa (pattern + NER) passa da F1 0.77 a **0.89**: GLiNER chiude i buchi del NER, portando address da 0 a F1 1.00 e il recall di `person_name` a 1.00.

7.1.0.0.3 Limiti. Il corpus è ridotto (poche decine di span) e sintetico: i valori vanno letti come **direzionali**, non come stime definitive, e la loro conferma richiede un corpus più ampio e realistico. Restano inoltre due punti aperti: la precisione di GLiNER su `person_name` è

Tabella 7.1: Validazione del livello di rilevamento sul corpus annotato (P, R, F1).

Configurazione	P	R	F1
Baseline regex (proprietaria)	0.93	0.68	0.79
Presidio — solo pattern/checksum	1.00	0.74	0.85
NER spaCy (it_core_news_lg)	0.17	0.05	0.08
NER GLiNER (gliner_multi_pii)	0.75	0.32	0.44
Presidio (pattern + NER spaCy)	0.75	0.79	0.77
Presidio (pattern + NER GLiNER)	0.89	0.89	0.89

ancora migliorabile (over-tagging occasionale), e l'inferenza GLiNER su CPU è più lenta di spaCy — accettabile, trattandosi di un modello *encoder* e non generativo, ma da tenere presente per il livello di rilevamento ricorrente (blocco B4).

Capitolo 8

Risultati e discussione

Capitolo 9

Sviluppi futuri

Capitolo 10

Studi effettuati

10.1 Uso AI in rapporto al suo impatto ambientale

Nella scelta della metodologia per la detection delle PII all'interno dei documenti, si è sollevato un problema non indifferente: che metodologia usare? I principali candidati (come espresso nella sezione [SEZIONE]) sono: rule-based, NLP / NER e generative AI. Come largamento approvato da innumerevoli studi, l'AI (specialmente usata in accoppiata con LLM) ottiene risultati praticamente perfetti, andando a scovare sfaccettature che alle altre metodologie sfuggono completamente. Specialmente, questa categorie di PII "perse per strada" sono quelle **legate al contesto, collegamenti multi-documenti e simili**.

Tuttavia, c'è un importante obiezione da tenere a mente: seppur la detection è sicuramente migliore usando LLM, il consumo energetico per il loro impiego è astronomicamente più alto rispetto ad utilizzare algoritmi deterministici o NER. [SEZIONE DI CONFRONTO CON NUMERI DA PAPER]

Alla luce di questi dati, è giusto fare un passo indietro e comprendere che...

10.2 Considerazioni su training

Capitolo 11

Punti aperti (*work in progress*)

Nota. Capitolo temporaneo di servizio. Riunisce, per comodità di discussione, le decisioni ancora da prendere sparse nel documento; ciascuna rimanda alla sezione in cui vive e dove andrà riportata la soluzione una volta chiusa. Va rimosso a lavoro concluso.

11.1 Decisioni progettuali aperte

11.1.1 Requisiti da confermare

11.1.1.0.1 PA-1 — Aggregazione delle PII frammentate. Va deciso se e in che misura il sistema debba ricondurre frammenti dispersi (in uno o più documenti) a un'unica entità coerente, e con quali criteri verificabili (co-occorrenza, similarità semantica, identificatori condivisi). Requisito attualmente marcato DA VALUTARE (cfr. §2.1.4).

11.1.1.0.2 PA-2 — Costruzione di una knowledge base delle entità. Va deciso se il sistema debba costruire, a partire dalle PII estratte, una rappresentazione strutturata delle entità (via NER) come base per il confronto col ROPA, e con quale livello di dettaglio. Requisito attualmente marcato DA VALUTARE (cfr. §2.1.5). Impatta direttamente su PA-1 e sullo *scope* complessivo (cfr. rischio di *scope creep*, §4.1.6).

11.1.2 Modello dati del ROPA

Voci già isolate come «punti aperti di progettazione» in §5.1.5, qui riunite.

11.1.2.0.1 PA-3 — Granularità della corrispondenza categorie–PII. Va fissato se una categoria di dati del ROPA mappi su un singolo *pii_type* o su un insieme. Una voce sbrigativa («dati anagrafici») corrisponde di norma a più categorie canoniche, il che suggerisce una relazione 1–N da formalizzare.

11.1.2.0.2 PA-4 — Allineamento nLPD–GDPR. Il modello parte dai campi CNIL (impianto GDPR); vanno annotate le differenze rilevanti dell'art. 12 LPD, in particolare soglie ed eccezioni per le PMI, che incidono su quali trattamenti debbano comparire nel registro.

11.1.2.0.3 PA-5 — ROPA non strutturato. Il modello assume un registro già compilato in forma tabellare. Va deciso se (e quanto) supportare un ROPA fornito come testo libero, restando entro il vincolo di non delegare all'AI la strutturazione complessiva del documento.

11.1.3 Persistenza del registro PII

11.1.3.0.1 Snapshot o event-logger Da decidere se il metodo per salvare le differenze tra versioni del FS analizzato seguano il paradigma degli snapshot o dell'event-logger.

11.1.3.0.2 PA-6 — Registrazione della «scomparsa» di una PII: esplicita o implicita? Quando una scansione non ritrova un'istanza confermata in precedenza, va deciso se scrivere attivamente una `PIIChange` di tipo `REMOVED` (esplicita, log autoesplicativo ma con confronto attivo a ogni scansione) o dedurre lo stato «rimossa» a posteriori (implicita, più semplice ma non autosufficiente). Ipotesi preferita: esplicita.

11.1.3.0.3 PA-7 — Significato dell'evento «modifica». Poiché il registro non salva i valori delle PII, un evento di «modifica» può riferirsi solo al cambio di **posizione** (`MOVED`) o a una **riclassificazione** del tipo. Va confermata l'ipotesi di non prevedere un generico `MODIFIED`, ma di scomporlo in variazioni dal significato preciso.

11.1.3.0.4 PA-8 — Vocabolario chiuso dei tipi di variazione. Si propone di formalizzare `change_type` come enumerazione chiusa (`NEW`, `CONFIRMED`, `MOVED`, `REMOVED`). Resta da confermare se il set è completo per tutti i casi d'uso previsti (dipende anche da PA-6).

11.1.4 Scelte implementative

11.1.4.0.1 PA-9 — Tecnologia di persistenza. Il modello concettuale è *technology-agnostic*; va scelta la tecnologia concreta tra base dati relazionale (es. `SQLite`, con *join* nativi) e persistenza su file semi-strutturati (`JSON/JSONL`, più leggera ma senza garanzie relazionali). La decisione è deliberatamente rimandata a valle della chiusura del modello dati (in particolare di PA-6, PA-7, PA-8) (cfr. §6.2).

11.1.4.0.2 PA-10 — Riutilizzo di Microsoft Presidio come livello di detection. Va deciso, su base empirica, se costruire il livello `regex/NER` sopra Presidio (incapsulato dietro il contratto `PIIDetector`) anziché scriverlo da zero. Il giudizio è affidato a una misurazione diretta (`precision/recall/F1`) sullo stesso corpus della baseline (cfr. §6.4).

11.1.5 Metodologia di detection

11.1.5.0.1 PA-11 — Strategia per il modello NER. Va scelto e motivato l'approccio tra: (A) zero-shot puro (GLiNER base), (B) modello pre-fine-tunato da terzi (es. *gliner-pii*), (C) fine-tuning leggero in proprio. Dai vincoli di progetto (*zero-training*, assenza di dati annotati, esecuzione locale) l'opzione C è da scartare a favore di A/B; manca la discussione strutturata con tabella di confronto nel Capitolo 5.

11.2 Contenuti ancora da redigere

Non decisioni, ma segnaposto e sezioni incomplete sparse nel testo, raccolti qui per non perderli.

- **TB-1** — Stato dell'arte: sezione «Definizione e inquadramento del problema» da riempire senza ripetere il contesto; definizione del problema tecnico dell'astrazione ancora allo stato di appunto (cfr. §3.2).
- **TB-2** — Stato dell'arte: la sezione sul *compliance checking* automatizzato è forse da ricollocare (cfr. §3.8 e dintorni).
- **TB-3** — Architettura: confronto strutturato e scelta del modello NER (tabella dei tre approcci di PA-11) da inserire nel Capitolo 5.
- **TB-4** — Architettura: diagramma ER in TikZ del modello di persistenza (*PIIInstance*, *PIIChange*, *Document*, *Scan*, *ProcessingActivity*), da produrre dopo la chiusura di PA-6/7/8 (cfr. §5.4.4).
- **TB-5** — Implementazione: roadmap implementativa e di validazione (baseline zero-shot GLiNER, integrazione modello pre-fine-tunato, eventuale esplorazione di fine-tuning) nel Capitolo 6.
- **TB-6** — Studi: sezione sull'impatto ambientale dell'AI da completare con i numeri dai paper di confronto e la conclusione (cfr. §10).
- **TB-7** — Studi: precisazione sul costo computazionale, distinguendo l'inferenza LLM ripetuta (esclusa per l'analisi massiva) dal fine-tuning *una tantum* di un encoder locale (cfr. §10.2).

Elenco delle figure

5.1	Architettura a macro-blocchi del sistema	34
5.2	Dettaglio del blocco B4: pipeline ibrida di rilevamento PII	35
5.3	Meccanismo di <i>human-in-the-loop</i> : nel normale lavoro con i documenti aziendali i dipendenti segnalano al DPO le PII in cui si imbattono; il DPO analizza le segnalazioni e arricchisce il ROPA o i parametri del sistema, migliorando il rilevamento. Il processo di analisi resta trasparente ai dipendenti, per i quali i file sono semplici documenti.	36
5.4	Diagramma ER del registro delle PII rilevate	45

Elenco delle tabelle

3.1 Confronto empirico degli approcci di PII detection basato su evidenze dalla letteratura. 25

5.1 Approcci valutati per la persistenza del registro delle PII. 42

5.2 Sintesi comparativa dei tre approcci di persistenza. 44

6.1 Criteri di selezione dello SLM e relative fonti. 60

6.2 Throughput (token/s), 4-bit GGUF su CPU [52]. 60

6.3 Occupazione di memoria: FP16 vs 4-bit (Q4) e RAM minima [52]. 61

6.4 Raccomandazione per task (fascia CPU). 62

7.1 Validazione del livello di rilevamento sul corpus annotato (P, R, F1). 68

Bibliografia

- [1] Shirley Radack. «Protecting Personally Identifiable Information». In: *Guide to Protecting the Confidentiality of Personally Identifiable Information (PII)*. NIST Special Publication (apr. 2010). URL: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-122.pdf>.
- [2] Parlamento europeo e Consiglio dell'Unione europea. *Regolamento (UE) 2016/679*. 27 Apr. 2016. URL: <http://data.europa.eu/eli/reg/2016/679/oj> (visitato il giorno 16/06/2026).
- [3] *RS 235.1 - Legge Federale Del 25 Settembre 2020 Sulla Protezione Dei Dati (LPD) / Fedlex*. URL: <https://www.fedlex.admin.ch/eli/cc/2022/491/it> (visitato il giorno 16/06/2026).
- [4] *IEEE-29148-SRS-LaTeX-Template/IEEE-29148-2018-SRS-Template.Tex at Main · Wxinx/IEEE-29148-SRS-LaTeX-Template*. GitHub. URL: <https://github.com/wxinx/IEEE-29148-SRS-LaTeX-Template/blob/main/IEEE-29148-2018-SRS-Template.tex> (visitato il giorno 13/06/2026).
- [5] Sona College of Technology, India et al. «A FRAMEWORK FOR ANALYSING UNSTRUCTURED DATA IN COMPUTING DEVICES». In: *ICTACT Journal on Soft Computing* 12.1 (1 ott. 2021), pp. 2464–2468. ISSN: 09766561, 22296956. DOI: 10.21917/ijsc.2021.0352. URL: <https://ictactjournals.in/IJSC/ArticleDetails?id=6604> (visitato il giorno 09/06/2026).
- [6] A. Makhambet e A. Moldagulova. «A Comparative Analysis of Machine Learning Methods for Personal Information Recognition (PII) in Unstructured Texts». In: *Computing & Engineering* 3.1 (31 mar. 2025), pp. 41–52. ISSN: 3080-3829. DOI: 10.51301/ce.2025.i1.07. URL: <https://ce.journal.satbayev.university/index.php/journal/article/view/1308> (visitato il giorno 09/06/2026).
- [7] Harshit Rajgarhia et al. *An Evaluation Study of Hybrid Methods for Multilingual PII Detection*. arXiv.org. 8 Ott. 2025. URL: <https://arxiv.org/abs/2510.07551v1> (visitato il giorno 13/06/2026).

- [8] Nagaraju Velishetty. «Personal Identifiable Information (PII) Detection and Identification for Fintech with AI and Text Analytics». Tesi di laurea mag. Dublin, National College of Ireland, 10 ago. 2024. 23 pp. URL: <https://norma.ncirl.ie/7037/> (visitato il giorno 09/06/2026).
- [9] Warty et al. «Systematic Literature Review on Named Entity Recognition: Approach, Method, and Application». In: *Statistics, Optimization & Information Computing* 12.4 (28 feb. 2024), pp. 907–942. ISSN: 2310-5070. DOI: 10.19139/soic-2310-5070-1631. URL: <https://iapress.org/index.php/soic/article/view/1631> (visitato il giorno 18/06/2026).
- [10] Jianliang Yang et al. «Exploring the Application of Large Language Models in Detecting and Protecting Personally Identifiable Information in Archival Data: A Comprehensive Study*». In: *2023 IEEE International Conference on Big Data (BigData)*. 2023 IEEE International Conference on Big Data (BigData). Sorrento, Italy: IEEE, 15 dic. 2023, pp. 2116–2123. ISBN: 979-8-3503-2445-7. DOI: 10.1109/BigData59044.2023.10386949. URL: <https://ieeexplore.ieee.org/document/10386949/> (visitato il giorno 17/06/2026).
- [11] Orlando Amaral et al. *NLP-based Automated Compliance Checking of Data Processing Agreements against GDPR*. 18 Giu. 2023. DOI: 10.48550/arXiv.2209.09722. arXiv: 2209.09722 [cs.SE]. URL: <http://arxiv.org/abs/2209.09722> (visitato il giorno 09/06/2026). Pre-pubblicato.
- [12] Sinan Gultekin et al. «An Energy-Based Comparative Analysis of Common Approaches to Text Classification in the Legal Domain». In: *AI, Machine Learning and Applications*. 27 Gen. 2024, pp. 31–41. DOI: 10.5121/csit.2024.140203. arXiv: 2311.01256 [cs.CL]. URL: <http://arxiv.org/abs/2311.01256> (visitato il giorno 19/06/2026).
- [13] Johannes Zschache e Tilman Hartwig. «Comparing Energy Consumption and Accuracy in Text Classification Inference». In: *Scientific Reports* 16.1 (17 apr. 2026), p. 12717. ISSN: 2045-2322. DOI: 10.1038/s41598-026-45023-0. arXiv: 2508.14170 [cs.CL]. URL: <http://arxiv.org/abs/2508.14170> (visitato il giorno 19/06/2026).
- [14] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. 2016. URL: <https://www.deeplearningbook.org/contents/ml.html> (visitato il giorno 21/06/2026).
- [15] Jacob Devlin et al. «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. NAACL-HLT 2019. A cura di Jill Burstein, Christy Doran e Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, giu. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: <https://aclanthology.org/N19-1423/> (visitato il giorno 21/06/2026).

- [16] Yongqin Xian et al. *Zero-Shot Learning – A Comprehensive Evaluation of the Good, the Bad and the Ugly*. 23 Set. 2020. DOI: 10.48550/arXiv.1707.00600. arXiv: 1707.00600 [cs.CV]. URL: <http://arxiv.org/abs/1707.00600> (visitato il giorno 21/06/2026). Pre-pubblicato.
- [17] Tom B. Brown et al. *Language Models Are Few-Shot Learners*. 22 Lug. 2020. DOI: 10.48550/arXiv.2005.14165. arXiv: 2005.14165 [cs.CL]. URL: <http://arxiv.org/abs/2005.14165> (visitato il giorno 21/06/2026). Pre-pubblicato.
- [18] Pritesh Jha. *PIIBench: A Unified Multi-Source Benchmark Corpus for Personally Identifiable Information Detection*. arXiv.org. 17 Apr. 2026. DOI: 10.5281/zenodo.19533263. URL: <https://arxiv.org/abs/2604.15776v1> (visitato il giorno 16/06/2026).
- [19] Luca Mainetti e Andrea Elia. «Detecting Personally Identifiable Information Through Natural Language Processing: A Step Forward». In: *ResearchGate* (20 mar. 2026). DOI: 10.3390/asi8020055. URL: https://www.researchgate.net/publication/390936945_Detecting_Personally_Identifiable_Information_Through_Natural_Language_Processing_A_Step_Forward (visitato il giorno 17/06/2026).
- [20] «Enterprise-Scale PII De-Identification with Microsoft Presidio Anonymizer: Architecture, Use Cases, and Best Practices». In: *ResearchGate* (3 mar. 2026). ISSN: 3050-9416. URL: https://www.researchgate.net/publication/399570056_Enterprise-Scale_PII_De-Identification_with_Microsoft_Presidio_Anonymizer_Architecture_Use_Cases_and_Best_Practices (visitato il giorno 17/06/2026).
- [21] Yanai Elazar et al. *What's In My Big Data?* 5 Mar. 2024. DOI: 10.48550/arXiv.2310.20707. arXiv: 2310.20707 [cs.CL]. URL: <http://arxiv.org/abs/2310.20707> (visitato il giorno 24/06/2026). Pre-pubblicato.
- [22] Minqi Jiang et al. *Generative Data Refinement: Just Ask for Better Data*. 11 Set. 2025. DOI: 10.48550/arXiv.2509.08653. arXiv: 2509.08653 [cs.LG]. URL: <http://arxiv.org/abs/2509.08653> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [23] Imed Keraghel, Stanislas Morbieu e Mohamed Nadif. *Recent Advances in Named Entity Recognition: A Comprehensive Survey and Comparative Study*. 20 Dic. 2024. DOI: 10.48550/arXiv.2401.10825. arXiv: 2401.10825 [cs.CL]. URL: <http://arxiv.org/abs/2401.10825> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [24] Noopur Zambare et al. *Towards Fair and Efficient De-identification: Quantifying the Efficiency and Generalizability of De-identification Approaches*. 27 Gen. 2026. DOI: 10.48550/arXiv.2602.15869. arXiv: 2602.15869 [cs.CL]. URL: <http://arxiv.org/abs/2602.15869> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [25] Ben. *Best Open Source Models for PII Redaction*. Grepture. 25 Mar. 2026. URL: <https://grepture.com/blog/best-open-source-models-pii-redaction> (visitato il giorno 25/06/2026).

- [26] Kushagra Mishra, Harsh Pagare e Kanhaiya Sharma. «A Hybrid Rule-Based NLP and Machine Learning Approach for PII Detection and Anonymization in Financial Documents». In: *Scientific Reports* 15.1 (2 lug. 2025), p. 22729. ISSN: 2045-2322. DOI: 10.1038/s41598-025-04971-9. URL: <https://www.nature.com/articles/s41598-025-04971-9> (visitato il giorno 25/06/2026).
- [27] Byamugisha Africano et al. «PII Detection in Low-Resource Languages Using Explainable Deep Learning Techniques». In: *Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing*. IC3 2024: 2024 Sixteenth International Conference on Contemporary Computing. Noida India: ACM, 8 ago. 2024, pp. 94–103. ISBN: 979-8-4007-0972-2. DOI: 10.1145/3675888.3676036. URL: <https://dl.acm.org/doi/10.1145/3675888.3676036> (visitato il giorno 25/06/2026).
- [28] Urchade Zaratiana, Ash Lewis e George Hurn-Maloney. *GLiNER2-PII: A Multilingual Model for Personally Identifiable Information Extraction*. arXiv.org. 11 Mag. 2026. URL: <https://arxiv.org/abs/2605.09973v1> (visitato il giorno 17/06/2026).
- [29] *Comparing Best NER Models For PII Identification*. 18 Nov. 2025. URL: <https://www.protecto.ai/blog/best-ner-models-for-pii-identification/> (visitato il giorno 25/06/2026).
- [30] *Knowledgator/Gliner-Pii-Base-v1.0 · Hugging Face*. 29 Gen. 2026. URL: <https://huggingface.co/knowledgator/gliner-pii-base-v1.0> (visitato il giorno 25/06/2026).
- [31] *Open Source PHI De-Identification: A Technical Review*. IntuitionLabs. URL: <https://intuitionlabs.ai/articles/open-source-phi-de-identification-tools> (visitato il giorno 25/06/2026).
- [32] Andres Felipe Zambrano et al. «De-Identifying Student Personally Identifying Information in Discussion Forum Posts with Large Language Models». In: *Information and Learning Sciences* 126.5–6 (7 apr. 2025), pp. 401–424. ISSN: 2398-5348. DOI: 10.1108/ILS-11-2024-0156. URL: <https://doi.org/10.1108/ILS-11-2024-0156> (visitato il giorno 25/06/2026).
- [33] Hao Yu et al. *Open, Closed, or Small Language Models for Text Classification?* 19 Ago. 2023. DOI: 10.48550/arXiv.2308.10092. arXiv: 2308.10092 [cs.CL]. URL: <http://arxiv.org/abs/2308.10092> (visitato il giorno 25/06/2026). Pre-pubblicato.
- [34] Lokesh Koli, Shubham Kalra e Karanpreet Singh. *Decoding Complexity: Intelligent Pattern Exploration with CHPDA (Context Aware Hybrid Pattern Detection Algorithm)*. 9 Feb. 2025. DOI: 10.48550/arXiv.2502.07815. arXiv: 2502.07815 [cs.CR]. URL: <http://arxiv.org/abs/2502.07815> (visitato il giorno 25/06/2026). Pre-pubblicato.

- [35] Noora Al Qahtani. *A Comparative Study of Supervised vs. Unsupervised Learning Techniques for Real-Time Intrusion Detection in Cybersecurity Networks*. ResearchGate. URL: https://www.researchgate.net/publication/396389868_A_Comparative_Study_of_Supervised_vs_Unsupervised_Learning_Techniques_for_Real-Time_Intrusion_Detection_in_Cybersecurity_Networks (visitato il giorno 20/06/2026).
- [36] Xichen Zhang et al. «Personally Identifiable Information Detection in Smart Edge Computing: A Robust Model Evaluation». In: *Knowledge-Based Systems* 328 (25 ott. 2025), p. 114153. ISSN: 0950-7051. DOI: 10.1016/j.knosys.2025.114153. URL: <https://www.sciencedirect.com/science/article/pii/S0950705125011943> (visitato il giorno 20/06/2026).
- [37] *Microsoft Presidio*. URL: <https://microsoft.github.io/presidio/> (visitato il giorno 22/06/2026).
- [38] Urchade Zaratiana et al. *GLiNER: Generalist Model for Named Entity Recognition Using Bidirectional Transformer*. 14 Nov. 2023. DOI: 10.48550/arXiv.2311.08526. arXiv: 2311.08526 [cs.CL]. URL: <http://arxiv.org/abs/2311.08526> (visitato il giorno 22/06/2026). Pre-pubblicato.
- [39] *Scrubadub*. 22 Giu. 2026. URL: <https://github.com/LeapBeyond/scrubadub> (visitato il giorno 22/06/2026).
- [40] *Overview of Varonis AI-Powered Data Discovery and Classification*. URL: <https://www.varonis.com/blog/ai-data-classification> (visitato il giorno 22/06/2026).
- [41] *Data Classification Buyer's Guide: How to Choose a Data Classification Solution*. URL: <https://www.varonis.com/blog/data-classification-buyers-guide> (visitato il giorno 16/06/2026).
- [42] *Varonis Software Pricing & Plans 2025: See Your Cost*. URL: <https://www.vendr.com/marketplace/varonis> (visitato il giorno 22/06/2026).
- [43] *BigID Data Sheet: GDPR Compliance*. Slideshare. URL: <https://www.slideshare.net/slideshow/bigid-data-sheet-gdpr-compliance-120471407/120471407> (visitato il giorno 16/06/2026).
- [44] *Privacy Suite*. BigID. URL: <https://bigid.com/privacy-suite/> (visitato il giorno 16/06/2026).
- [45] *Detecting Personal Names in Texts: Technical Whitepaper*. Technical whitepaper. Limassol, Cipro: PII-Tools Ltd., 2024.
- [46] *PII Tools | Discover, Analyze & Remediate Sensitive Data Anywhere*. PII Tools. URL: <https://pii-tools.com/> (visitato il giorno 16/06/2026).

- [47] Michiel Overeem et al. «An Empirical Characterization of Event Sourced Systems and Their Schema Evolution – Lessons from Industry». In: *Journal of Systems and Software* 178 (ago. 2021), p. 110970. ISSN: 01641212. DOI: 10.1016/j.jss.2021.110970. arXiv: 2104.01146 [cs.SE]. URL: <http://arxiv.org/abs/2104.01146> (visitato il giorno 09/07/2026).
- [48] Greg Young. «CQRS Documents by Greg Young». In: ().
- [49] Alexandra Sasha Luccioni, Yacine Jernite e Emma Strubell. «Power Hungry Processing: Watts Driving the Cost of AI Deployment?» In: *The 2024 ACM Conference on Fairness Accountability and Transparency*. 3 Giu. 2024, pp. 85–99. DOI: 10.1145/3630106.3658542. arXiv: 2311.16863 [cs.LG]. URL: <http://arxiv.org/abs/2311.16863> (visitato il giorno 19/06/2026).
- [50] Zhenyan Lu et al. *Small Language Models: Survey, Measurements, and Insights*. 26 Feb. 2025. DOI: 10.48550/arXiv.2409.15790. arXiv: 2409.15790 [cs.CL]. URL: <http://arxiv.org/abs/2409.15790> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [51] Qingyu Song et al. *A Systematic Evaluation of On-Device LLMs: Quantization, Performance, and Resources*. 16 Mar. 2026. DOI: 10.48550/arXiv.2505.15030. arXiv: 2505.15030 [cs.LG]. URL: <http://arxiv.org/abs/2505.15030> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [52] Kyrylo S Malakhov. «Deploying LLMs on CPU-only Environments with Llama.Cpp Library Set: MedLocalGPT Project Case». In: ().
- [53] Uygar Kurt. *Which Quantization Should I Use? A Unified Evaluation of Llama.Cpp Quantization on Llama-3.1-8B-Instruct*. 11 Gen. 2026. DOI: 10.48550/arXiv.2601.14277. arXiv: 2601.14277 [cs.LG]. URL: <http://arxiv.org/abs/2601.14277> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [54] Sönke Tenckhoff, Mario Koddenbrock e Erik Rodner. *LLMStructBench: Benchmarking Large Language Model Structured Data Extraction*. 16 Feb. 2026. DOI: 10.48550/arXiv.2602.14743. arXiv: 2602.14743 [cs.CL]. URL: <http://arxiv.org/abs/2602.14743> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [55] Nikolai Skripko. *Instruction-Following Evaluation in Function Calling for Large Language Models*. 22 Set. 2025. DOI: 10.48550/arXiv.2509.18420. arXiv: 2509.18420 [cs.AI]. URL: <http://arxiv.org/abs/2509.18420> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [56] *Ibm-Granite/Granite-3.0-Language-Models*. IBM Granite, 17 lug. 2026. URL: <https://github.com/ibm-granite/granite-3.0-language-models> (visitato il giorno 18/07/2026).

- [57] Gemma Team et al. *Gemma 3 Technical Report*. 25 Mar. 2025. DOI: 10.48550/arXiv.2503.19786. arXiv: 2503.19786 [cs.CL]. URL: <http://arxiv.org/abs/2503.19786> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [58] *Meta-Llama/Llama-3.2-1B-Instruct* · Hugging Face. 6 Dic. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct> (visitato il giorno 18/07/2026).
- [59] Josip Tomo Licardo e Nikola Tankovic. *Performance Trade-offs of Optimizing Small Language Models for E-Commerce*. 24 Ott. 2025. DOI: 10.48550/arXiv.2510.21970. arXiv: 2510.21970 [cs.AI]. URL: <http://arxiv.org/abs/2510.21970> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [60] Haolin Zhang e Jeff Huang. *Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference*. 9 Mag. 2025. DOI: 10.48550/arXiv.2505.06461. arXiv: 2505.06461 [cs.DC]. URL: <http://arxiv.org/abs/2505.06461> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [61] Agatsya Yadav e Renta Chintala Bhargavi. «Optimizing LLMs Using Quantization for Mobile Execution». In: vol. 1654. 2026, pp. 330–339. DOI: 10.1007/978-3-032-06697-8_33. arXiv: 2512.06490 [cs.LG]. URL: <http://arxiv.org/abs/2512.06490> (visitato il giorno 18/07/2026).
- [62] Qwen et al. *Qwen2.5 Technical Report*. 3 Gen. 2025. DOI: 10.48550/arXiv.2412.15115. arXiv: 2412.15115 [cs.CL]. URL: <http://arxiv.org/abs/2412.15115> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [63] An Yang et al. *Qwen2 Technical Report*. 10 Set. 2024. DOI: 10.48550/arXiv.2407.10671. arXiv: 2407.10671 [cs.CL]. URL: <http://arxiv.org/abs/2407.10671> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [64] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 30 Ago. 2024. DOI: 10.48550/arXiv.2404.14219. arXiv: 2404.14219 [cs.CL]. URL: <http://arxiv.org/abs/2404.14219> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [65] Microsoft et al. *Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs*. 7 Mar. 2025. DOI: 10.48550/arXiv.2503.01743. arXiv: 2503.01743 [cs.CL]. URL: <http://arxiv.org/abs/2503.01743> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [66] Loubna Ben Allal et al. *SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model*. 4 Feb. 2025. DOI: 10.48550/arXiv.2502.02737. arXiv: 2502.02737 [cs.CL]. URL: <http://arxiv.org/abs/2502.02737> (visitato il giorno 18/07/2026). Pre-pubblicato.

- [67] Cosimo Galeone et al. *When Correct Isn't Usable: Improving Structured Output Reliability in Small Language Models*. 4 Mag. 2026. DOI: 10.48550/arXiv.2605.02363. arXiv: 2605.02363 [cs.CL]. URL: <http://arxiv.org/abs/2605.02363> (visitato il giorno 18/07/2026). Pre-pubblicato.
- [68] Md Romyull Islam et al. *Characterizing and Understanding Energy Footprint and Efficiency of Small Language Model on Edges*. 7 Nov. 2025. DOI: 10.48550/arXiv.2511.11624. arXiv: 2511.11624 [cs.DC]. URL: <http://arxiv.org/abs/2511.11624> (visitato il giorno 18/07/2026). Pre-pubblicato.